

Survival regression with accelerated failure time model in XGBoost

Avinash Barnwal*
Stony Brook University
avinashbarnwal123@gmail.com

Hyunsu Cho
NVIDIA
phcho@nvidia.com

Toby Hocking
Northern Arizona University
toby.hocking@nau.edu

August 24, 2021

Abstract

Survival regression is used to estimate the relation between time-to-event and feature variables, and is important in application domains such as medicine, marketing, risk management and sales management. Nonlinear tree based machine learning algorithms as implemented in libraries such as XGBoost, scikit-learn, LightGBM, and CatBoost are often more accurate in practice than linear models. However, existing state-of-the-art implementations of tree-based models have offered limited support for survival regression. In this work, we implement loss functions for learning accelerated failure time (AFT) models in XGBoost, to increase the support for survival modeling for different kinds of label censoring. We demonstrate with real and simulated experiments the effectiveness of AFT in XGBoost with respect to a number of baselines, in two respects: generalization performance and training speed. Furthermore, we take advantage of the support for NVIDIA GPUs in XGBoost to achieve substantial speedup over multi-core CPUs. To our knowledge, our work is the first implementation of AFT that utilizes the processing power of NVIDIA GPUs. Starting from the 1.2.0 release, the XGBoost package natively supports the AFT model. The addition of AFT in XGBoost has had significant impact in the open source community, and a few statistics packages now utilize the XGBoost AFT model.

1 Introduction

Survival analysis is a prominent subfield of statistics, where the goal is to model time duration to a given event (e.g. death). Given the nature of time-to-event data, labels may not be

*The author gratefully acknowledges Google for supporting him via Google Summer of Code 2019.

completely observed and only censored labels are given. For data points whose label is censored, the exact label y is not known but only a range $(\underline{y}, \bar{y})$ that contains it. The topic has drawn a large body of research literature in the last few decades. See [46] for a general survey.

The Cox proportional hazards (Cox-PH) model [14] is one of the most commonly used models in survival analysis. The model estimates the hazard function $h(t)$, which is defined to be the likelihood of the event occurring at time t given that no event has occurred before time t . The Cox-PH model is of form $h(t, \mathbf{x}) = h_0(t) \exp(\langle \mathbf{w}, \mathbf{x} \rangle)$, where the baseline hazard function $h_0(t)$ depends only on time and the features $\mathbf{x}$ have multiplicative effects on h . Given the parameters $\mathbf{w}$, it is clear which of the normalized features $\mathbf{x}$ has the largest effect on survival. However, it is non-trivial to predict time-to-event $\hat{y}(\mathbf{x})$ in the Cox-PH model. We would need to estimate the baseline hazard function $h_0(t)$ using a non-parametric estimator known as Breslow’s estimator [2, 7]. The computation of Breslow’s estimator requires access to the full training data and is computationally expensive for big data.

The Accelerated Failure Time (AFT) model is another well known method for survival analysis, although perhaps less often used than Cox-PH. We choose to explore AFT in this paper for two primary reasons. First, we would like to not only analyze model parameters (coefficients) but also perform predictive analysis. While Cox-PH gives relative importance of features, it does not yield a usable prediction $\hat{y}$ easily [2]. With the AFT model, we can predict unknown labels using only the fitted parameters and a feature vector. Second, the AFT model may provide a better fit when proportional hazard assumption does not hold [17].

Miller [31] proposed the AFT model for the first time, and later Buckley and James [8] refined it to obtain an asymptotically consistent estimator using the least squares approach. Khan and Shaw [27] combined AFT with adaptive and weighted elastic nets to enable variable selection from high-dimensional data. See [13, 47] for overviews on the topic of AFT models.

Tree-based models have shown better performance than linear models in terms of detecting complex and nonlinear patterns in the feature variables. The gradient boosting algorithm [19] fits an additive ensemble of decision trees in a stepwise fashion to greedily optimize a general class of loss functions $\ell(y, \hat{y})$. Gradient boosting is widely used due to its simplicity and predictive performance. The algorithm produces an ensemble of decision trees and exhibits many desirable properties as a statistical model, such as being slow to overfitting and having asymptotic convergence guarantees [10, 48]. Gradient boosting is versatile, as it can optimize a general class of loss function $\ell(y, \hat{y})$ where y represents the true label and $\hat{y}$ the predicted label. It has been successfully used in classification [18], document ranking [9], structured prediction [12] and other applications. Today, there are several scalable, efficient software packages that

implement gradient boosting, including XGBoost [11], LightGBM [26], Scikit-Learn [34], and Catboost [36].

XGBoost is a fast implementation of gradient boosting that speeds up convergence by using the second-order partial derivative of the loss function. XGBoost is able to integrate with a variety of programming environments such as R and Python and integrates with frameworks for distributed computing, such as Dask and Apache Spark. Integration of AFT with XGBoost therefore makes survival analysis easier in the big data setting.

There are a few previous implementations of survival analysis in tree based models. Schmid and Hothorn [38] used boosting framework for AFT and considers the negative log-likelihood as loss function. There are also other tree based survival models such as Random Survival Trees [24], Cox-Boosting [4], Bagging Survival Trees [22], Scikit-Survival [35] and Cox-PH in XGBoost [30]. Most of the models are limited to right-censored outcomes. Maximum Margin Interval Trees [16] support interval-censored labels.

Survival analysis is broadly useful in a variety of applications, such as survival prediction of cancer patients [45], customer churn [43], credit scoring [15], failure times of mechanical systems [3, 39]. However, binary machine learning classifiers have been often used where survival methodology is applicable, due to concerns about predictive accuracy [28]. For example, Vaid et al. [42] used XGBoost binary classifiers to predict whether COVID-19 patients will develop complications in a given time frame, achieving substantially better AUC-ROC and AUC-PR than generalized linear models. While binary classifiers may provide for a state-of-art predictive accuracy, one loses flexibility that comes from directly modeling time duration to events: one is forced to decide predetermined duration(s) where an event is to occur or not. AFT in XGBoost addresses these challenges in the following two ways. First, the model is able to capture nonlinear patterns in the data. Second, the model readily produces survival time estimates; to compute predictions, we only need the fitted model parameters and a feature vector.

Summary of novel contributions. We propose a novel adaptation of the AFT model to integrate with XGBoost. Our implementation supports all modes of label censoring, including interval-censoring. We run experiments with real and simulated data sets to demonstrate the generalization performance of the AFT model in XGBoost. Furthermore, we are able to accelerate training by using XGBoost’s built-in support for NVIDIA GPUs.

2 AFT in XGBoost

The original AFT model takes the following form:

$$\ln Y = \langle \mathbf{w}, \mathbf{x} \rangle + \sigma Z \quad (1)$$

where $\mathbf{x}$ represents the input features, $\mathbf{w}$ the coefficients, Y the survival time (the output label), and Z a random variable of a known probability distribution. Both Y and Z are random variables. Note that this model is a generalized form of a linear regression model $Y = \langle \mathbf{w}, \mathbf{x} \rangle$. In order to make AFT work with gradient boosting, we revise the model as follows:

$$\ln Y = \mathcal{T}(\mathbf{x}) + \sigma Z \quad (1')$$

where $\mathcal{T}(\mathbf{x})$ represents the output from the decision tree ensemble, given input $\mathbf{x}$.

2.1 Derivation of AFT loss function

XGBoost optimizes a twice-differentiable convex loss function $\ell(\cdot, \cdot)$ in its second-order method of gradient boosting [11]. We will now define a suitable loss function ℓ_{AFT} to represent the AFT model. Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ denote the training data, and let $Y_1, \dots, Y_n$ denote random variables i.i.d. with the distribution for Y . Let f_Y and F_Y denote the probability density function (PDF) and the cumulative distribution function (CDF) for Y , respectively. The likelihood function for $\mathcal{D}$ is the product of probability densities f_Y for individual data points:

$$L(\mathcal{D}) = \mathbb{P}[Y_1 = y_1, \dots, Y_n = y_n] = \prod_{i=1}^n \mathbb{P}[Y_i = y_i] = \prod_{i=1}^n f_Y(y_i) \quad (2)$$

As commonly done in machine learning literature, we maximize log likelihood instead:

$$\ln L(\mathcal{D}) = \sum_{i=1}^n \ln \mathbb{P}[Y_i = y_i] = \sum_{i=1}^n \ln f_Y(y_i) \quad (2')$$

Since we do not know y_i for some data points, due to label censoring, we revise the likelihood function (2') to take account of the censored labels:

$$\begin{aligned} \ln L(\mathcal{D}) &= \underbrace{\sum \ln \mathbb{P}[Y_i = y_i]}_{\text{uncensored label}} + \underbrace{\sum \ln \mathbb{P}[\underline{y}_i \leq Y_i \leq \bar{y}_i]}_{\text{censored label with } y_i \in [\underline{y}_i, \bar{y}_i]} \\ &= \underbrace{\sum \ln f_Y(y_i)}_{\text{uncensored label}} + \underbrace{\sum \ln (F_Y(\bar{y}_i) - F_Y(\underline{y}_i))}_{\text{censored label with } y_i \in [\underline{y}_i, \bar{y}_i]} \end{aligned}$$

Table 1: List of label censoring types

Label censoring	Lower bound ($\underline{y}_i$)	Upper bound ($\overline{y}_i$)
Right-censored	Finite non-negative	$+\infty$
Left-censored	0	Finite non-negative
Interval-censored	Finite non-negative	Finite non-negative

where $\underline{y}_i$ and $\overline{y}_i$ are lower and upper bounds for the label y_i , respectively. Note that $\overline{y}_i$ may be infinity, to indicate right-censored labels. See Table 1 for full list of censoring types. We are now ready to define the loss function ℓ_{AFT} .

Definition 1 (Loss function for AFT survival regression).

$$\ell_{\text{AFT}}(y, \mathcal{T}(\mathbf{x})) = \begin{cases} -\ln f_Y(y) & \text{if } y \text{ is not censored} \\ -\ln (F_Y(\overline{y}) - F_Y(\underline{y})) & \text{if } y \text{ is censored with } y \in [\underline{y}, \overline{y}] \end{cases} \quad (3)$$

Under this definition, the sum of losses $\sum_{i=1}^n \ell(y_i, \mathcal{T}(\mathbf{x}_i))$ over the training data is identical to $-\ln L(\mathcal{D})$. Since we only know distribution of Z (not of Y), we use the following lemma:

Lemma 1 ((1.27) of [5]). *Let Y and Z be random variables. If $Y = g(Z)$ with a monotone increasing function $g(\cdot)$ that is suitably smooth, the PDF and CDF of Y are expressed in terms of the PDF and CDF of Z as follows:*

$$f_Y(y) = f_Z(g^{-1}(y)) \cdot \frac{d}{dy} g^{-1}(y) \quad F_Y(y) = F_Z(g^{-1}(y)) \quad (4)$$

We apply Lemma 1 to (3) with $g(Z) = \exp(\mathcal{T}(\mathbf{x}) + \sigma Z)$ to get the following formula for ℓ_{AFT} :

Definition 2 (Loss function for AFT survival regression, in terms of known PDF and CDF).

$$\ell_{\text{AFT}}(y, \mathcal{T}(\mathbf{x})) = \begin{cases} -\ln \left[f_Z(s(y)) \cdot \frac{1}{\sigma y} \right] & \text{if } y \text{ is not censored} \\ -\ln [F_Z(s(\overline{y})) - F_Z(s(\underline{y}))] & \text{if } y \text{ is censored with } y \in [\underline{y}, \overline{y}] \end{cases} \quad (3')$$

where f_Z and F_Z are given by Table 2 and $s(y) = s(y, \mathcal{T}(\mathbf{x})) = (\ln y - \mathcal{T}(\mathbf{x}))/\sigma$ is a link function.

See Figure 1 for a geometric representation. Since the prediction $\mathcal{T}(\mathbf{x})$ from the tree ensemble model approximates the log survival time $\ln y$, we use the link function $s(y, \mathcal{T}(\mathbf{x})) = (\ln y - \mathcal{T}(\mathbf{x}))/\sigma$ in Definition 2 as a convenient measure for the distance between the prediction

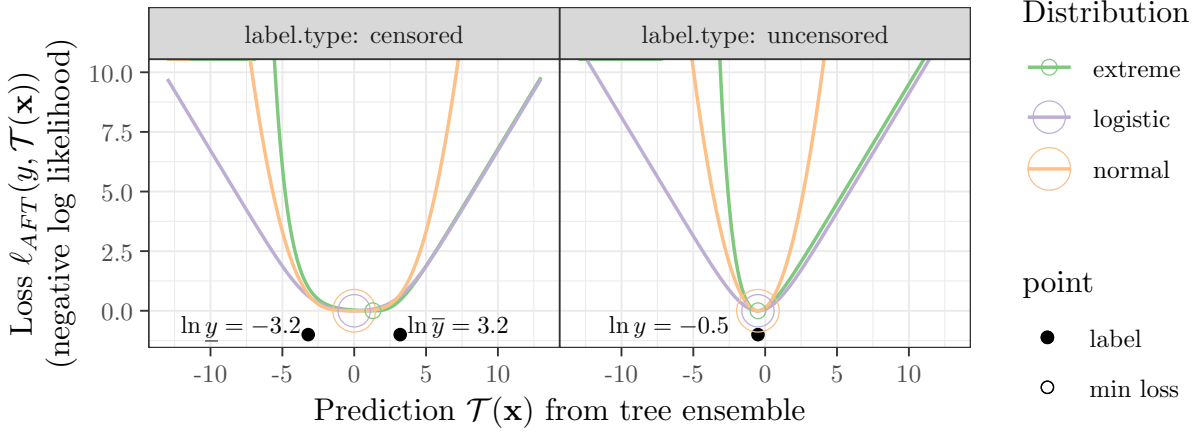


Figure 1: Geometric interpretation of the Accelerated Failure Time (AFT) loss (colored curves), using three distributions (normal, logistic, extreme) with scale parameter $\sigma = 1$. Log survival times are shown using solid black dots, and loss function minima are shown using open colored circles. Note that the prediction $\mathcal{T}(\mathbf{x})$ from the tree ensemble model is in the same scale as the log survival time $\ln y$. **Left:** for censored data the loss function is defined as the negative log of the difference of cumulative distribution function values. The example shown has finite upper and lower limits, for which the minimum of the logistic/normal loss occurs at the midpoint between the two limits, whereas it occurs at a greater value for the extreme distribution. **Right:** for uncensored data the loss function is defined as the negative log of the density function, so the normal loss is the usual square loss with symmetric quadratic tails. The logistic loss has symmetric linear tails, whereas the asymmetric extreme loss has a linear upper tail and an exponential lower tail.

and the log survival time. Although s is a function of two variables, we will use $s(y)$ as a shorthand for $s(y, \mathcal{T}(\mathbf{x}))$ to save space.

2.2 Gradient and hessian of the AFT loss

The gradient boosting algorithm in XGBoost uses the gradient and hessian of the loss function, which are first and second partial derivatives of $\ell(y, \mathcal{T}(\mathbf{x}))$ with respect to the second input $\mathcal{T}(\mathbf{x})$. To express partial derivatives in a concise manner, define a single-letter variable $u = \mathcal{T}(\mathbf{x})$ as an alias for the output from the tree ensemble model. The gradient and hessian of the AFT loss function are as follows¹:

¹For left- and right-censored labels, let $f_Z(-\infty) = f_Z(\infty) = 0$ and $F_Z(-\infty) = 0, F_Z(+\infty) = 1$.

Table 2: Probability distributions for Z

Distribution	PDF ($f_Z(z)$)	CDF ($F_Z(z)$)	$f'_Z(z)$	$f''_Z(z)$
Normal	$\frac{\exp(-z^2/2)}{\sqrt{2\pi}}$	$\frac{1}{2} \left(1 + \operatorname{erf} \left(\frac{z}{\sqrt{2}} \right) \right)$	$-zf_Z(z)$	$(z^2 - 1)f_Z(z)$
Logistic	$\frac{e^z}{(1 + e^z)^2}$	$\frac{e^z}{1 + e^z}$	$\frac{f_Z(z)(1 - e^z)}{1 + e^z}$	$\frac{f_Z(z)(e^{2z} - 4e^z + 1)}{(1 + e^z)^2}$
Extreme ¹	$e^z e^{-\exp z}$	$1 - e^{-\exp z}$	$(1 - e^z)f_Z(z)$	$(e^{2z} - 3e^z + 1)f_Z(z)$

¹ Also known as the Gumbel (minimum) distribution. See [40].

Definition 3 (Gradient and hessian of AFT loss).

$$\left. \frac{\partial \ell_{\text{AFT}}}{\partial u} \right|_{y,u} = \begin{cases} \frac{f'_Z(s(y))}{\sigma f_Z(s(y))} & \text{if } y \text{ is not censored} \\ \frac{f_Z(s(\bar{y})) - f_Z(s(\underline{y}))}{\sigma [F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]} & \text{if } y \text{ is censored with } y \in [\underline{y}, \bar{y}] \end{cases} \quad (5)$$

$$\left. \frac{\partial^2 \ell_{\text{AFT}}}{\partial u^2} \right|_{y,u} = \begin{cases} -\frac{f_Z(s(y))f''_Z(s(y)) - f'_Z(s(y))^2}{\sigma^2 f_Z(s(y))^2} & \text{if } y \text{ is not censored} \\ -\frac{[F_Z(s(\bar{y})) - F_Z(s(\underline{y}))][f'_Z(s(\bar{y})) - f'_Z(s(\underline{y}))] + [f_Z(s(\bar{y})) - f_Z(s(\underline{y}))]^2}{\sigma^2 [F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]^2} & \text{if } y \text{ is censored} \end{cases} \quad (6)$$

where f'_Z and f''_Z are the first and second derivatives of the PDF f_Z , respectively, and $s(y) = (\ln y - u)/\sigma$ is defined the same way as in Definition 2. See Table 2 to look up f'_Z and f''_Z for the three known distributions.

Proof. The first- and second-order partial derivatives of ℓ_{AFT} may be derived using basic rules of Calculus. Consult Appendix C for the full proof. $\square$

2.3 Regularization for the AFT loss, to avoid numerical instabilities

The equations (3'), (5), and (6) may suffer from numerical instabilities when the prediction $u = \mathcal{T}(\mathbf{x})$ from the tree ensemble model is far away from the true log survival time $\ln y \in [\ln \underline{y}, \ln \bar{y}]$. There are three causes for the numerical instabilities:

- Zero passed to the logarithm function $\ln(\cdot)$: the difference term $F_Z(s(\bar{y})) - F_Z(s(\underline{y}))$ in (3') becomes nearly zero when the prediction $u = \mathcal{T}(\mathbf{x})$ is far away from the true log survival time $\ln y \in [\ln \underline{y}, \ln \bar{y}]$. Passing zero to the logarithm results in a NAN (Not-a-Number).
- Zero denominator: the denominators in (5), and (6) for censored data contain the difference term $F_Z(s(\bar{y})) - F_Z(s(\underline{y}))$, which can be nearly zero for the same reason as above. Zero denominator results in a NAN (Not-a-Number).
- Large number passed to the exponential function $\exp(\cdot)$: the PDF and CDF of the logistic and extreme distributions contain the exponential function. Since 64-bit floating-point variables in a C++ program hold up to 10^{308} , the exponential function yields in an INF (infinity) even for moderately large inputs.

Refer to the IEEE 754 standard [23] to learn more about the special representations of floating-point values, such as INFs and NANs. In order to eliminate INFs and NANs, we apply regularization in two places.

2.3.1 Regularization for the loss function (3')

We replace the difference term $F_Z(s(\bar{y})) - F_Z(s(\underline{y}))$ with $\epsilon = 10^{-12}$ whenever the difference term is smaller than ϵ .

2.3.2 Regularization for the gradient (5) and the hessian (6)

We explicitly define values of the gradient and hessian at the limit $u \rightarrow \pm\infty$. Whenever a numerical instability is detected due to u being far away from the true log survival time, we set the gradient and hessian values according to Table 3. We clip all gradients to ± 15 , as large gradients cause numerical difficulties in XGBoost. In addition, we always assign 10^{-16} hessian value to all data points, because data points with zero hessian are ignored by XGBoost. Regularization forces XGBoost to consider every data point to a certain extent.

Table 3: Specification of the gradient and hessian values ($\partial\ell/\partial u$, $\partial^2\ell/\partial u^2$) as $u \rightarrow \pm\infty$.

Distribution for Z	Label type	Uncensored		Right-censored	
		$u \rightarrow -\infty$	$u \rightarrow +\infty$	$u \rightarrow -\infty$	$u \rightarrow +\infty$
Normal	$\partial\ell/\partial u$	-15	15	-15	0
	$\partial^2\ell/\partial u^2$	$1/\sigma^2$	$1/\sigma^2$	$1/\sigma^2$	10^{-16}
Logistic	$\partial\ell/\partial u$	$-1/\sigma$	$1/\sigma$	$-1/\sigma$	0
	$\partial^2\ell/\partial u^2$	10^{-16}	10^{-16}	10^{-16}	10^{-16}
Extreme	$\partial\ell/\partial u$	-15	$1/\sigma$	-15	0
	$\partial^2\ell/\partial u^2$	15	10^{-16}	15	10^{-16}

Distribution for Z	Label type	Left-censored		Interval-censored	
		$u \rightarrow -\infty$	$u \rightarrow +\infty$	$u \rightarrow -\infty$	$u \rightarrow +\infty$
Normal	$\partial\ell/\partial u$	0	15	-15	15
	$\partial^2\ell/\partial u^2$	10^{-16}	$1/\sigma^2$	$1/\sigma^2$	$1/\sigma^2$
Logistic	$\partial\ell/\partial u$	0	$1/\sigma$	$-1/\sigma$	$1/\sigma$
	$\partial^2\ell/\partial u^2$	10^{-16}	10^{-16}	10^{-16}	10^{-16}
Extreme	$\partial\ell/\partial u$	0	$1/\sigma$	-15	$1/\sigma$
	$\partial^2\ell/\partial u^2$	10^{-16}	10^{-16}	15	10^{-16}

3 Experiments

3.1 Effectiveness of AFT for interval-censored data

We measure the effectiveness of the XGBoost AFT model for interval-censored data. We define the accuracy metric for data with interval-censored labels as follows:

$$\text{Accuracy}(\mathcal{D}) = \frac{|\{i : \mathcal{T}(\mathbf{x}_i) \in [\ln \underline{y}_i, \ln \overline{y}_i]\}|}{|\mathcal{D}|}, \quad (7)$$

i.e. the fraction of data points for which the prediction from the tree ensemble model falls between the lower and upper bounds for the true log survival time.

The XGBoost AFT model is compared to three baselines:

survreg Un-regularized linear model with AFT loss functions [40] on principal components, with the number of components selected using cross-validation.

penaltyLearning L1-regularized linear model with squared hinge loss [37], with the degree of L1 regularization selected by cross-validation.

MMIT Max Margin Interval Trees [16], which generalizes the well-known Classification and Regression Tree (CART) algorithm of [6] to censored outputs. The tree depth is selected using cross-validation.

We perform nested cross-validation to estimate the generalization performance of the model as well as the hyperparameter search procedure. We use 5-fold internal cross-validation to evaluate multiple hyperparameter combinations. Each combination is judged using the mean validation accuracy over the 5 folds. (The mean validation accuracy is also used to determine the number of boosting rounds.) We then perform 4-fold external cross-validation to quantify the generalization performance of the training procedure, as follows: we fit a new model using the best hyperparameter combinations, using all data points except the held-out test set. The test accuracy is evaluated with the held-out test set. For hyperparameter search, we run 100 trials in the random search; see Section 3.3 for details.

Experiments in Sections 3.1.1 and 3.1.2 were conducted using a workstation with one Intel Core i7-7800X CPU (3.50 GHz, 6 cores) and two DDR4 RAMs (16 GB each, 2133 MHz), running Ubuntu 18.04 LTS.

3.1.1 Interval-censored data from supervised changepoint detection problems

To test our algorithm in real data sets with interval-censored outputs, we consider a benchmark of supervised peak detection problems in genomic ChIP-seq data [21, 37]. Each of the ten data sets in Table 4 corresponds to a set of high-throughput DNA sequencing experiments. Expert biologists manually labeled each data set to indicate locations with and without peaks; these labels are used to compute the peak detection error rate. The goal of the ChIP-seq peak detection is to automatically locate peaks given a new DNA sequence, such that peak detection error rate is minimal. Each data set is named like `H3K4me3_PGP_immune`:

- The first field (`H3K4me3`) identifies the assay type. Consult the McGill Epigenomics Mapping Centre website² for the full list of assay types and their meaning.
- The second field (`PGP`) is the initials of the expert biologist who provided the labels.
- The last field (`immune`) identifies a sample set. Consult [21] for more information.

A univariate signal is computed for each sample by computing coverage frequency of aligned DNA sequence reads at each positions of the reference genome sequence. To detect peaks in the univariate signal, we use an changepoint detection algorithm with learned penalty functions [37]. Briefly, the penalty parameter λ of the changepoint detection algorithm controls the number of distinct segments/peaks detected. It is possible to compute a range of optimal penalty values $[\underline{\lambda}_*, \overline{\lambda}_*]$ that are optimal in terms of the expert-provided labels; that is, setting λ to any value in $[\underline{\lambda}_*, \overline{\lambda}_*]$ will minimize the label error rate. We cast the peak detection problem into an interval-censored regression problem as follows. The univariate signal is

²<https://epigenomesportal.ca/edcc/index.html>

Table 4: Dimensions of ChIP-seq data sets and descriptive statistics. The log lambda values given below are averages over the data set.

	Data set	Rows	Columns	min.log.lambda	max.log.lambda
(1)	ATAC_JV_adipose	465	36	8.581	10.470
(2)	CTCF_TDH_ENCODE	182	36	10.246	12.643
(3)	H3K27ac-H3K4me3_TDHAM_BP	2008	36	7.674	9.641
(4)	H3K27ac_TDH_some	95	36	9.318	10.365
(5)	H3K36me3_AM_immune	420	36	8.955	12.723
(6)	H3K27me3_RL_cancer	171	36	14.332	16.192
(7)	H3K27me3_TDH_some	43	36	10.617	11.389
(8)	H3K36me3_TDH_ENCODE	78	36	8.147	9.634
(9)	H3K36me3_TDH_immune	84	36	10.939	13.003
(10)	H3K36me3_TDH_other	40	36	9.742	12.389

used to compute a feature vector $\mathbf{x} \in \mathbb{R}^{36}$ that stores various summary statistics, such as percentiles, of the signal. The range of optimal penalty values $[\underline{\lambda}_*, \overline{\lambda}_*]$ is taken to be an interval-censored label.

We pre-process the data as follows: we apply the exponential function $\exp(\cdot)$ to the output labels `min.log.lambda` and `max.log.lambda` to obtain the non-negative interval-censored labels `min.lambda` and `max.lambda`. Then we remove all feature columns that either 1) had at least one missing value or 2) had zero variance.

Figure 2a shows the generalization performance (test accuracy) and run time of XGBoost and the baseline packages. XGBoost exhibits competitive generalization performance on par with SurvReg, penaltyLearning, and MMIT. In addition, XGBoost is fast: its run time approaches that of SurvReg and penaltyLearning and significantly smaller than that of MMIT. Considering that SurvReg and penaltyLearning are linear models and MMIT nonlinear, the run-time performance speaks to the efficiency of XGBoost.

3.1.2 Synthetic interval-censored data generated from known distributions

Drouin [16] generated synthetic interval-censored data based on three kind of features: sine, absolute and linear. It has a mix of nonlinear and linear features having 200 samples and 20 features in each data set. We extend it with three more data sets having more complex nonlinear features. We use a random number generator to generate interval-censored data as follows.

First, generate the feature vectors $\mathbf{x} \in \mathbb{R}^{20}$ by sampling from the uniform distribution $U([0, 10])$. Second, draw 10 values randomly from the normal distribution $\mathcal{N}(f(\mathbf{x}), 0.3)$ where the mean is determined with a function $f : \mathbb{R}^{20} \rightarrow \mathbb{R}$ that maps the feature vector $\mathbf{x}$ to a real

value. Third, out of the 10 values, choose the smallest as the lower bound $\underline{y}$ and the largest as the upper bound $\bar{y}$. Lastly, add a small noise to both the interval bounds by sampling a value from $\mathcal{N}(0, 0.2)$ and adding it to $\underline{y}$ and $\bar{y}$.

Each generated data set was named after the choice for f :

- `simulated.sin`: $f(\mathbf{x}) = \sin(x_1)$
- `simulated.abs`: $f(\mathbf{x}) = |x_1 - 5|$
- `simulated.linear`: $f(\mathbf{x}) = x_1/5$
- `simulated.model.1`: $f(\mathbf{x}) = x_1x_2 + x_3^2 - x_4x_7 + x_8x_{10} - x_6^2$
- `simulated.model.2`: $f(\mathbf{x}) = -\sin(2x_1) + x_2^2 + x_3 - \exp(-x_4)$
- `simulated.model.3`: $f(\mathbf{x}) = x_1 + 3x_3^2 - 2\exp(-x_5)$

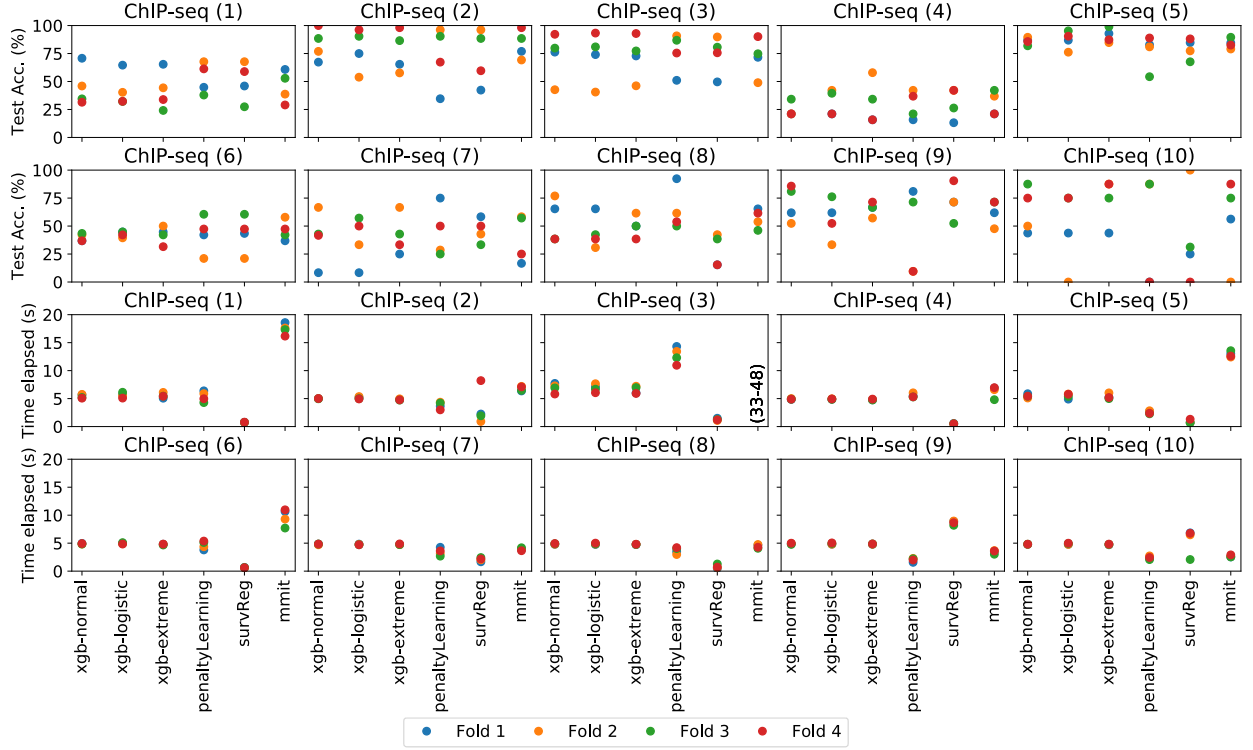
We compare the performance of `penaltyLearning`, `survReg`, `MMIT` and `XGBoost` on test data of size 100. As in Section 3.1.1, we perform nested cross-validation to estimate the generalization performance of the model as well as the hyperparameter search procedure; this time, we use 5 folds for both the outer and inner cross-validation. We reproduce the behavior in [16], where nonlinear models like `mmit` better capture nonlinear patterns in simulated data than linear models do. In Figure 2b, both `XGBoost` and `mmit` exhibit superior generalization performance (test accuracy) compared to the linear models, `SurvReg` and `penaltyLearning`. The additional run-time incurred by the nonlinear models is compensated by higher test accuracy. The difference between `XGBoost` and `mmit` is more pronounced when we look at the three simulated data we added apart from those from [16]. `XGBoost` runs faster than `mmit`, up to 3x, and shows higher test accuracy. In particular, for `simulated.model.3`, `XGBoost` achieves 58.5% mean test accuracy, whereas `mmit` achieves 18%.

3.2 Effectiveness of AFT on right-censored data

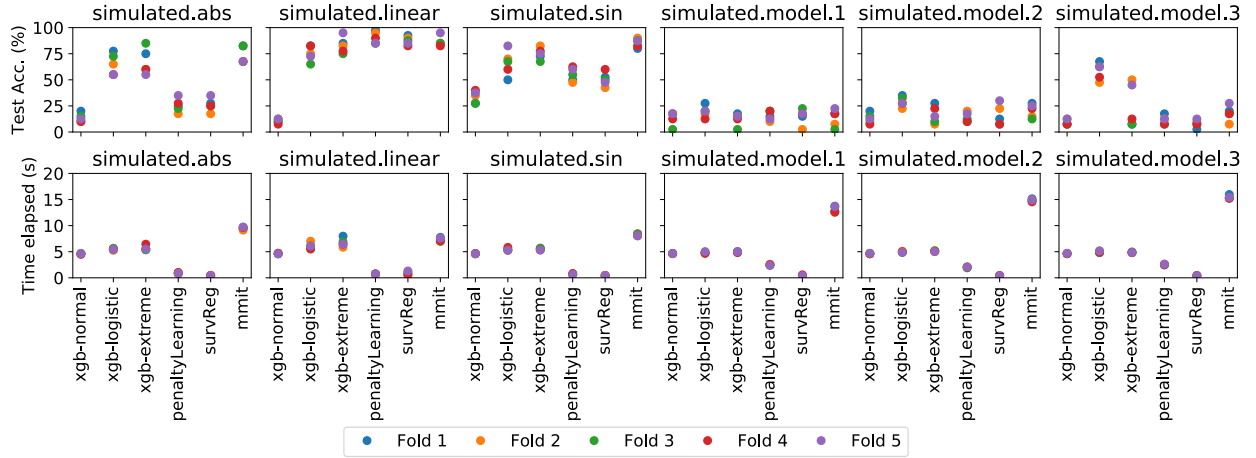
We measure the effectiveness of the `XGBoost` AFT model for right-censored data using Uno’s C-statistics [41], which is a modified form of Harell’s Concorance Index [20]. Uno’s C-statistics is an unbiased nonparametric estimator for the following ranking metric:

$$C = \mathbb{P}[\mathcal{T}(\mathbf{x}_i) < \mathcal{T}(\mathbf{x}_j) | y_i < y_j, y_i < \tau] \quad (8)$$

The τ constant is chosen judiciously in order to truncate outlier labels when estimating C . In this experiment, we set τ to the 80th percentile of the observed survival time. We use the implementation of Uno’s C-statistics from the `Scikit-Survival` package [35].



(a) ChIP-seq data from Table 4.



(b) Simulated data.

Figure 2: Experimental results from Section 3.1: test accuracy and run time

The XGBoost AFT model is compared to two baselines:

XGBoost-Cox Cox-PH model from the XGBoost package [30], enabled by setting configuration `objective='survival:cox'`.

Scikit-Survival Cox-PH linear model from the Scikit-Survival package [35]

As in Section 3.1, we use nested cross-validation to assess the generalization performance of the model as well as the hyperparameter search procedure. For hyperparameter search, we run 100 trials in the random search; see Section 3.3 for details.

3.2.1 Synthetic data with a mix of uncensored and right-censored labels

Using a random number generator, we generate synthetic data consisting a mix of uncensored and right-censored labels, as follows³:

1. Draw a feature vector $\mathbf{x} \in \mathbb{R}^{20}$ from the uniform distribution $U([0, 1])$.
2. Draw the “risk score” $r \in \mathbb{R}$ from the normal distribution $\mathcal{N}(f(\mathbf{x}), 0.3)$ where $f(\mathbf{x}) = x_1 + 3x_3^2 - 2\exp(-x_5)$ is a nonlinear map.
3. Draw u from the uniform distribution $U([0, 1])$.
4. Compute the ground-truth survival time $y = -\ln u / (h_0 h^r)$, where $h_0 = 0.1$ is the baseline hazard and $h = 2.0$ is the hazard ratio.
5. Simulate the effect of random censoring by drawing cutoff value c from the uniform distribution $U([0, C])$, where C is suitably chosen to induce censoring for a set fraction of data points. If $y \geq c$, the label is right-censored and we set the label range $[\underline{y}, \bar{y}] = [c, +\infty)$. If $y < c$, the label is not censored and we set $[\underline{y}, \bar{y}] = [y, y]$.

Repeat the steps to generate 1000 data points. The experiment result with this method of data generation is shown with label `data_gen=coxph` in Figure 3. When 20% the labels are (right-)censored, the Cox-PH model from Scikit-Survival produces slightly higher C-statistics metric than XGBoost models. On the other hand, with greater amount of censoring (50%, 80%), the test C-statistics for XGBoost-AFT and XGBoost-CoxPH are similar to the test C-statistics for Scikit-Survival’s Cox-PH.

We now generate data with a mix of uncensored and right-censored labels using a different method.

³This method is adapted from a tutorial on the Scikit-Survival package’s website [35].

1. Draw a feature vector $\mathbf{x} \in \mathbb{R}^{20}$ from the uniform distribution $U([0, 1])$.
2. Draw the “risk score” $r \in \mathbb{R}$ from the normal distribution $\mathcal{N}(f(\mathbf{x}), 0.3)$ where $f(\mathbf{x}) = x_1 + 3x_3^2 - 2\exp(-x_5)$ is a nonlinear map.
3. Compute the ground-truth survival time $y = \exp(-r)$.
4. Simulate the effect of random censoring by drawing cutoff value c from the uniform distribution $U([0, C])$. This step is analogous to Step 5 of the previous recipe.

Repeat the steps to generate 1000 data points. The experiment result with this method of data generation is shown with label `data_gen=aft` in Figure 3. When 20% of the labels are censored, XGBoost-AFT with the normal distribution produces slightly higher C-statistics metric than all other models. On the other hand, when 50% of the labels were censored, there is no clear winner; XGBoost-AFT and XGBoost-CoxPH produce similar test C-statistics as Scikit-Survival’s Cox-PH. With 80% censoring, Scikit-Survival’s Cox-PH produces the highest test C-statistics, and XGBoost-AFT and XGBoost-Cox are slightly behind. In all settings, XGBoost-AFT runs 2-3 $\times$ faster than XGBoost-Cox-PH or Scikit-Survival’s Cox-PH.

3.3 Effect of hyperparameters on model performance

To measure how sensitive the generalization performance is to the choice of hyperparameters, we try an array of approaches for selecting hyperparameters. There are 6 relevant hyperparameters: `learning_rate`, `max_depth`, `min_child_weight`, `reg_alpha`, `reg_lambda`, and `aft_loss_distribution_scale`⁴. The following methods are considered:

Grid search We select one or two hyperparameters out of the six and exhaustively enumerate all combinations using the grid in Table 5. If a hyperparameter is not chosen for the grid search, we assign a default value as follows: `learning_rate` = 0.1, `max_depth` = 6, `min_child_weight` = 1.0, `reg_alpha` = 0.001, `reg_lambda` = 1.0, `aft_loss_distribution_scale` = 1.0.

Random search We use Optuna [1] to randomly sample hyperparameter combinations from the search space (Table 5). All six hyperparameters are sampled. Each search is run for 100 or 1000 combinations.

Baseline (do nothing) Choose default values for all hyperparameters and perform no search.

⁴ σ in (1)

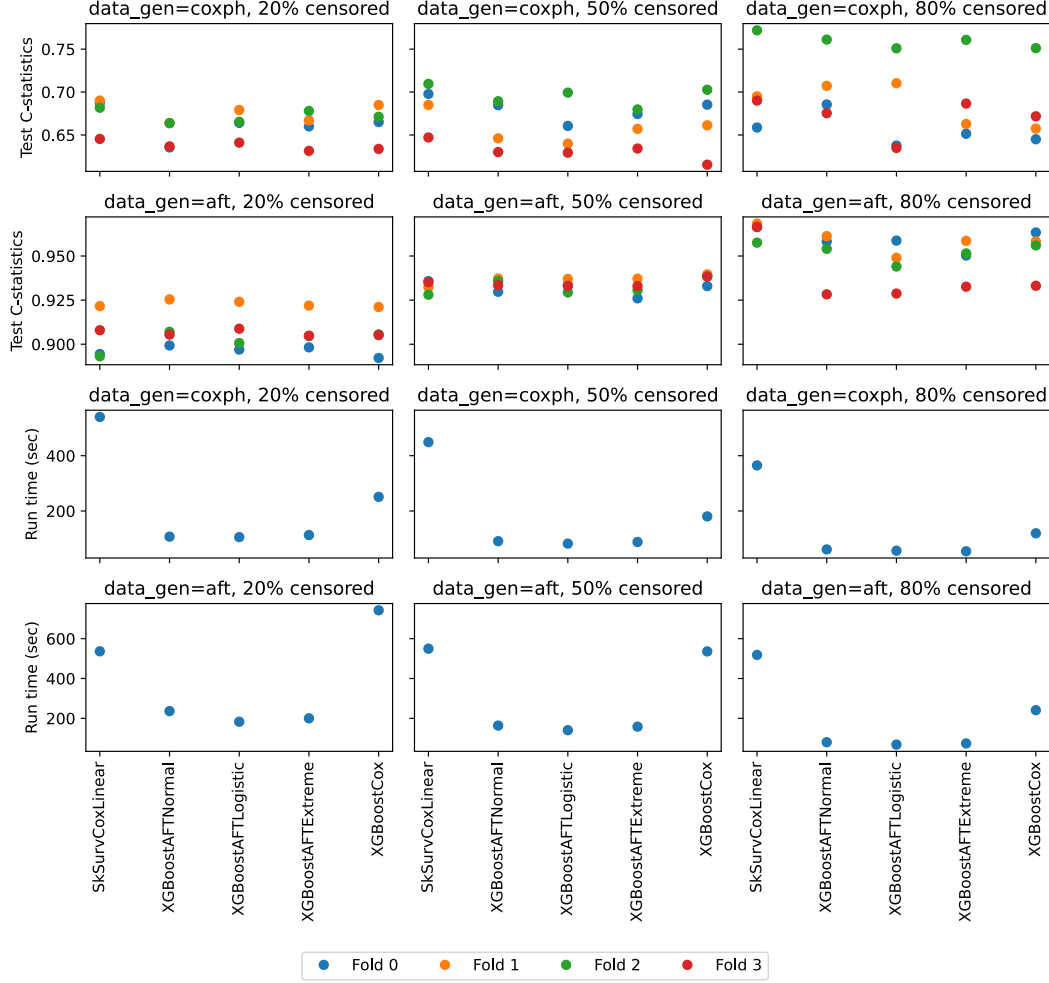


Figure 3: Experimental results from Section 3.2: test accuracy and run time

For the grid search, we try all possible ways of choosing one or two hyperparameters out of six. The generalization performance is measured in the aggregate with test accuracy.

As in Section 3.1.1, we perform nested cross-validation to estimate the generalization performance of the model as well as the hyperparameter search procedure. We use 4 and 5 folds for the outer and inner cross-validation, respectively. We used data sets from Sections 3.1.1 and 3.1.2.

In order to perform lots of hyperparameter search in a short amount of time, Amazon Web Services (AWS) is used to launch parallel jobs, in order to evaluate many hyperparameter search strategies. The manager EC2 instance launches hundreds of worker EC2 instances and then submits commands to execute via SSH. To ensure that all software dependencies are available to the experiment code as well as our XGBoost code, we package our code in a Docker container and host the container on Elastic Container Registry (ECR). The workers then pull the latest container image from ECR. The experiment is logged to an S3 bucket.

Table 5: Search space for hyperparameters

Hyperparameter	Search grid
<code>learning_rate</code>	0.001, 0.01, 0.1, 1.0
<code>max_depth</code>	2, 3, 4, 5, 6, 7, 8, 9, 10
<code>min_child_weight</code>	0.001, 0.1, 1.0, 10.0, 100.0
<code>reg_alpha</code>	0.001, 0.01, 0.1, 1.0, 10.0, 100.0
<code>reg_lambda</code>	0.001, 0.01, 0.1, 1.0, 10.0, 100.0
<code>aft_loss_distribution_scale</code>	0.5, 0.8, 1.1, 1.4, 1.7, 2.0
Hyperparameter	Distribution for random search
<code>learning_rate</code>	log uniform in [0.001, 1.0]
<code>max_depth</code>	integers in [2, 10]
<code>min_child_weight</code>	log uniform in [0.001, 100.0]
<code>reg_alpha</code>	log uniform in [0.001, 100.0]
<code>reg_lambda</code>	log uniform in [0.001, 100.0]
<code>aft_loss_distribution_scale</code>	uniform in [0.5, 2.0]

In all runs, the random search with 1000 trials gives the highest validation accuracy. However, high validation accuracy does not lead to high test accuracy. The grid search with one or two hyperparameters, where the number of trials is fewer than 100, yields higher test accuracy than the random search with 1000 trials. Refer to Appendix D to find the results for all datasets and hyperparameter search methods. When it comes to improving aggregate measure of generalization, test accuracy, it suffices to try 100 hyperparameter combinations; it does not make much difference in test accuracy to try more than 100 combinations.

3.4 Efficient model fitting with NVIDIA GPUs

XGBoost is able to utilize NVIDIA GPUs to accelerate its gradient boosting algorithm [32, 33]. We port the AFT loss function so that it can run on NVIDIA GPUs. To test the performance, we generate a synthetic data set with 20 million samples, by duplicating 100000 times⁵ the data `simulated.model.3` from Section 3.1.2. We then fit 5 XGBoost models using the 5 cross-validation folds. Each model is trained using the best hyperparameters found in Section 3.1.2. Table 6 shows the timing results. In all folds, the GPU fits the model 6.1-6.7× faster than the CPU.

We used NVIDIA Quadro® RTX 8000 with CUDA 10.2. The GPU has 4608 cores divided into 72 streaming multiprocessors and 48 GB GDDR6 memory.

⁵The duplicated rows got the same fold assignment as their originals, so that the fraction of data points belonging to each cross-validation fold remains the same.

Table 6: Comparing performance of CPU and GPU using the 20 million synthetic data

Test Fold ID	# boosting rounds	Run time (sec)		
		CPU	GPU	Speedup
1	52	50.72	8.36	6.1×
2	149	150.33	22.49	6.7×
3	53	54.07	8.52	6.3×
4	81	81.33	12.44	6.5×
5	83	92.48	14.13	6.5×

4 Limitations

In this section we discuss two limitations of our current approach: sensitivity to hyperparameters and prediction of survival curves. First, even though Section 3.3 shows that the test accuracy is not very sensitive to the choice of hyperparameters, sensitivity to hyperparameters manifests itself in other ways. Vieira et al. [44] present an example where two XGBoost AFT models that were fit with different values for the hyperparameter `aft_loss_distribution_scale` (and all the other hyperparameters the same) achieved nearly identical C-index metric on a validation data set but produced highly different mean survival time on the same validation set. Aggregate metrics fail to account for this phenomenon.

In addition, the XGBoost software package lacks some tools that are often used in the literature of survival analysis, such as survival curve and confidence interval computation. The survival curve is defined as, for each time point t , the proportion of the population for which the event would complete by t . Although the XGBoost predict method only computes a point estimate (mean) of the survival time for each individual in the population, interested users could also compute a survival curve using the chosen AFT distribution and scale parameter. A concern with such survival curves is that they may not be well-calibrated. In many applications, statistical models should be well calibrated to produce accurate probabilistic predictions, which in turn let us to accurately quantify the uncertainty around the given prediction.

It is possible to mitigate the aforementioned limitations. After our code became part of the XGBoost package on August 2020, a follow-up work [44] addressed the shortcomings mentioned above, via model stacking. The authors created a new package XGBSE that built on top of the XGBoost AFT model, where the output of the XGBoost model is used as an input to a second model that is amenable to probability calibration, such as logistic regression or nearest neighbor. A survival curve is obtained by fitting a Kaplan-Meier estimator [25] from the output of the second model. With this approach of model stacking, the authors

were able to obtain well-calibrated survival curves that are not sensitive to the choice of hyperparameters. In short, XGBSE capitalized on existing strengths of XGBoost AFT while mitigating its limitations. The authors state that they chose to build on XGBoost AFT, because of its state-of-the-art discriminating power and generalization performance as given in test metrics.

5 Conclusion

We implemented the Accelerated Failure Time model in XGBoost, a widely used library for gradient boosting. Using real and simulated data sets, we show that AFT in XGBoost show competitive generalization performance and run-time efficiency, both for interval-censored and right-censored data. XGBoost gives superior generalization capacity compared to linear baselines, `survReg` and `penaltyLearning`, and runs faster than the nonlinear baseline, `mmit`. Furthermore, AFT in XGBoost is able to take advantage of many capabilities of the ecosystem of XGBoost, such as support for NVIDIA GPUs. A future work may take advantage of integration of XGBoost into distributed computing frameworks such as Apache Spark and Dask.

Since August 2020, when our work became part of the XGBoost package, it has enabled follow-up work in open-source statistical software. Already, packages such as XGBSE and MLR3 [29, 44] take advantage of the support for AFT in XGBoost. In particular, XGBSE was built on top of our work and addressed the major shortcomings (see Section 4). Usage of our software indicates real-world relevance and impact of our contribution.

Acknowledgements

This work was supported by Google Summer of Code 2019.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. “Optuna: a Next-Generation Hyperparameter Optimization Framework”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD ’19. Anchorage, AK, USA: Association for Computing Machinery, 2019, 2623–2631. ISBN: 9781450362016. DOI: 10.1145/3292500.3330701. URL: <https://doi.org/10.1145/3292500.3330701>.
- [2] Paul D. Allison. *Survival analysis using SAS: a practical guide*. SAS Institute, 2010.

- [3] Abbas Barabadi, Javad Barabady, and Tore Markeset. “Application of accelerated failure model for the oil and gas industry in Arctic region”. In: *2010 IEEE International Conference on Industrial Engineering and Engineering Management*. 2010, pp. 2244–2248.
- [4] Harald Binder and Martin Schumacher. “Allowing for Mandatory Covariates in Boosting Estimation of Sparse High-Dimensional Survival Models”. In: *BMC bioinformatics* 9 (Feb. 2008), p. 14. DOI: 10.1186/1471-2105-9-14.
- [5] Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Berlin, Heidelberg: Springer-Verlag, 2006. ISBN: 0387310738.
- [6] Leo Breiman, Jerome Friedman, Charles J Stone, and Richard A Olshen. *Classification and regression trees*. CRC press, 1984.
- [7] Norman E. Breslow. “Discussion on Professor Cox’s Paper”. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 34.2 (1972), pp. 202–220. DOI: 10.1111/j.2517-6161.1972.tb00900.x. eprint: <https://rss.onlinelibrary.wiley.com/doi/pdf/10.1111/j.2517-6161.1972.tb00900.x>. URL: <https://rss.onlinelibrary.wiley.com/doi/abs/10.1111/j.2517-6161.1972.tb00900.x>.
- [8] Jonathan Buckley and Ian James. “Linear Regression with Censored Data”. In: *Biometrika* 66.3 (1979), pp. 429–436. ISSN: 00063444. URL: <http://www.jstor.org/stable/2335161>.
- [9] Chris J.C. Burges. *From RankNet to LambdaRank to LambdaMART: An Overview*. Tech. rep. MSR-TR-2010-82. June 2010. URL: <https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambdamart-an-overview/>.
- [10] Peter Bühlmann and Torsten Hothorn. “Boosting Algorithms: Regularization, Prediction and Model Fitting”. In: *Statistical Science* 22.4 (Nov. 2007), pp. 477–505. DOI: 10.1214/07-STS242. URL: <https://doi.org/10.1214/07-STS242>.
- [11] Tianqi Chen and Carlos Guestrin. “XGBoost: a Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD ’16. San Francisco, California, USA: Association for Computing Machinery, 2016, 785–794. ISBN: 9781450342322. DOI: 10.1145/2939672.2939785. URL: <https://doi.org/10.1145/2939672.2939785>.

- [12] Tianqi Chen, Sameer Singh, Ben Taskar, and Carlos Guestrin. “Efficient Second-Order Gradient Boosting for Conditional Random Fields”. In: *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Guy Lebanon and S. V. N. Vishwanathan. Vol. 38. Proceedings of Machine Learning Research. San Diego, California, USA: PMLR, May 2015, pp. 147–155. URL: <http://proceedings.mlr.press/v38/chen15b.html>.
- [13] Sy Chiou, Sangwook Kang, and Jun Yan. “Fitting Accelerated Failure Time Models in Routine Survival Analysis with R Package aftgee”. In: *Journal of Statistical Software, Articles* 61.11 (2014), pp. 1–23. ISSN: 1548-7660. DOI: 10.18637/jss.v061.i11. URL: <https://www.jstatsoft.org/v061/i11>.
- [14] D. R. Cox. “Regression Models and Life-Tables”. In: *Journal of the Royal Statistical Society. Series B (Methodological)* 34.2 (1972), pp. 187–220. ISSN: 00359246. URL: <http://www.jstor.org/stable/2985181>.
- [15] Lore Dirick, Gerda Claeskens, and Bart Baesens. “Time to default in credit scoring using survival analysis: a benchmark study”. In: *Journal of the Operational Research Society* 68.6 (2017), pp. 652–665. DOI: 10.1057/s41274-016-0128-9. eprint: <https://doi.org/10.1057/s41274-016-0128-9>. URL: <https://doi.org/10.1057/s41274-016-0128-9>.
- [16] Alexandre Drouin, Toby Hocking, and Francois Laviolette. “Maximum Margin Interval Trees”. In: *Advances in Neural Information Processing Systems 30*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Curran Associates, Inc., 2017, pp. 4947–4956. URL: <http://papers.nips.cc/paper/7080-maximum-margin-interval-trees.pdf>.
- [17] Alfensi Faruk. “The comparison of proportional hazards and accelerated failure time models in analyzing the first birth interval survival data”. In: *Journal of Physics: Conference Series* 974 (Mar. 2018), p. 012008. DOI: 10.1088/1742-6596/974/1/012008. URL: <https://doi.org/10.1088/1742-6596/974/1/012008>.
- [18] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. “Additive logistic regression: a statistical view of boosting (With discussion and a rejoinder by the authors)”. In: *The Annals of Statistics* 28.2 (Apr. 2000), pp. 337–407. DOI: 10.1214/aos/1016218223. URL: <https://doi.org/10.1214/aos/1016218223>.
- [19] Jerome H. Friedman. “Greedy Function Approximation: a Gradient Boosting Machine”. In: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232. ISSN: 00905364. URL: <http://www.jstor.org/stable/2699986>.

- [20] Frank E. Harrell Jr., Robert M. Califf, David B. Pryor, Kerry L. Lee, and Robert A. Rosati. “Evaluating the Yield of Medical Tests”. In: *JAMA: The Journal of the American Medical Association* 247.18 (May 1982), pp. 2543–2546. ISSN: 0098-7484. DOI: 10.1001/jama.1982.03320430047030. URL: <https://doi.org/10.1001/jama.1982.03320430047030>.
- [21] Toby Dylan Hocking, Patricia Goerner-Potvin, Andreanne Morin, Xiaojian Shao, Tomi Pastinen, and Guillaume Bourque. “Optimizing ChIP-seq peak detectors using visual labels and supervised machine learning”. In: *Bioinformatics* 33.4 (Nov. 2016), pp. 491–499. ISSN: 1367-4803. URL: <https://doi.org/10.1093/bioinformatics/btw672>.
- [22] Torsten Hothorn, Berthold Lausen, Axel Benner, and Martin Radespiel-Tröger. “Bagging survival trees”. In: *Statistics in Medicine* 23.1 (2004), pp. 77–91. DOI: 10.1002/sim.1593. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/sim.1593>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/sim.1593>.
- [23] IEEE. “IEEE Standard for Floating-Point Arithmetic”. In: *IEEE Std 754-2019 (Revision of IEEE 754-2008)* (2019), pp. 1–84.
- [24] Hemant Ishwaran, Udaya B. Kogalur, Eugene H. Blackstone, and Michael S. Lauer. “Random survival forests”. In: *The Annals of Applied Statistics* 2.3 (Sept. 2008), pp. 841–860. DOI: 10.1214/08-AOAS169. URL: <https://doi.org/10.1214/08-AOAS169>.
- [25] E. L. Kaplan and Paul Meier. “Nonparametric Estimation from Incomplete Observations”. In: *Journal of the American Statistical Association* 53.282 (1958), pp. 457–481. ISSN: 01621459. URL: <http://www.jstor.org/stable/2281868>.
- [26] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”. In: *Advances in Neural Information Processing Systems 30*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Curran Associates, Inc., 2017, pp. 3146–3154. URL: <http://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree.pdf>.
- [27] Md Hasinur Khan and J. Ewart H. Shaw. “Variable Selection for Survival Data with a Class of Adaptive Elastic Net Techniques”. In: *Statistics and Computing* 26.3 (May 2016), 725–741. ISSN: 0960-3174. DOI: 10.1007/s11222-015-9555-8. URL: <https://doi.org/10.1007/s11222-015-9555-8>.
- [28] Håvard Kvamme, Ørnulf Borgan, and Ida Scheel. “Time-to-Event Prediction with Neural Networks and Cox Regression”. In: *Journal of Machine Learning Research* 20.129 (2019), pp. 1–30. URL: <http://jmlr.org/papers/v20/18-424.html>.

- [29] Michel Lang, Martin Binder, Jakob Richter, Patrick Schratz, Florian Pfisterer, Stefan Coors, Quay Au, Giuseppe Casalicchio, Lars Kotthoff, and Bernd Bischl. “mlr3: A modern object-oriented machine learning framework in R”. In: *Journal of Open Source Software* (2019). DOI: 10.21105/joss.01903. URL: <https://joss.theoj.org/papers/10.21105/joss.01903>.
- [30] Scott M. Lundberg, Gabriel G. Erion, and Su-In Lee. *Consistent Individualized Feature Attribution for Tree Ensembles*. Mar. 7, 2019. arXiv: 1802.03888 [cs.LG].
- [31] Rupert G. Miller. “Least Squares Regression with Censored Data”. In: *Biometrika* 63.3 (1976), pp. 449–464. ISSN: 00063444. URL: <http://www.jstor.org/stable/2335722>.
- [32] Rory Mitchell and Eibe Frank. “Accelerating the XGBoost algorithm using GPU computing”. In: *PeerJ Computer Science* 3 (2017), e127.
- [33] Rong Ou. *Out-of-Core GPU Gradient Boosting*. 2020. arXiv: 2005.09148 [cs.LG].
- [34] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [35] Sebastian Pölsterl. “scikit-survival: a Library for Time-to-Event Analysis Built on Top of scikit-learn”. In: *Journal of Machine Learning Research* 21.212 (2020), pp. 1–6. URL: <http://jmlr.org/papers/v21/20-729.html>.
- [36] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. “CatBoost: unbiased boosting with categorical features”. In: *Advances in Neural Information Processing Systems 31*. Ed. by S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett. Curran Associates, Inc., 2018, pp. 6638–6648. URL: <http://papers.nips.cc/paper/7898-catboost-unbiased-boosting-with-categorical-features.pdf>.
- [37] Guillem Rigauill, Toby Hocking, Jean-Philippe Vert, and Francis Bach. “Learning sparse penalties for change-point detection using max margin interval regression”. In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. 2013, pp. 172–180.
- [38] Matthias Schmid and Torsten Hothorn. “Flexible boosting of accelerated failure time models”. In: *BMC Bioinformatics* 9 (2008), pp. 269–269.

- [39] Gian A. Susto, Andrea Schirru, Simone Pampuri, Seán McLoone, and Alessandro Beghi. “Machine Learning for Predictive Maintenance: a Multiple Classifier Approach”. In: *IEEE Transactions on Industrial Informatics* 11.3 (2015), pp. 812–820.
- [40] Terry M Therneau. *A Package for Survival Analysis in S*. version 2.38. 2015. URL: <https://CRAN.R-project.org/package=survival>.
- [41] Hajime Uno, Tianxi Cai, Michael J. Pencina, Ralph B D’Agostino, and Lee-Jen Wei. “On the C-statistics for evaluating overall adequacy of risk prediction procedures with censored survival data”. In: *Statistics in Medicine* 30.10 (2011), pp. 1105–1117. DOI: 10.1002/sim.4154. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3079915/>.
- [42] Akhil Vaid, Sulaiman Somani, Adam J. Russak, Jessica K. De Freitas, Fayzan F. Chaudhry, Ishan Paranjpe, Kipp W. Johnson, Samuel J. Lee, Riccardo Miotto, Felix Richter, Shan Zhao, Noam D. Beckmann, Nidhi Naik, Arash Kia, Prem Timsina, Anuradha Lala, Manish Paranjpe, Eddy Golden, Matteo Danieletto, Manbir Singh, Dara Meyer, Paul F. O’Reilly, Laura Huckins, Patricia Kovatch, Joseph Finkelstein, Robert M. Freeman, Edgar Argulian, Andrew Kasarskis, Bethany Percha, Judith A. Aberg, Emilia Bagiella, Carol R. Horowitz, Barbara Murphy, Eric J. Nestler, Eric E. Schadt, Judy H. Cho, Carlos Cordon-Cardo, Valentin Fuster, Dennis S. Charney, David L. Reich, Erwin P. Bottinger, Matthew A. Levin, Jagat Narula, Zahi A. Fayad, Allan C. Just, Alexander W. Charney, Girish N. Nadkarni, and Benjamin S. Glicksberg. “Machine Learning to Predict Mortality and Critical Events in a Cohort of Patients With COVID-19 in New York City: model Development and Validation”. In: *Journal of Medical Internet Research* 22.11 (Nov. 2020), e24018. ISSN: 1438-8871. DOI: 10.2196/24018. URL: <https://www.jmir.org/2020/11/e24018>.
- [43] Dirk Van den Poel and Bart Larivière. “Customer attrition analysis for financial services using proportional hazard models”. In: *European Journal of Operational Research* 157.1 (2004). Smooth and Nonsmooth Optimization, pp. 196–217. ISSN: 0377-2217. DOI: [https://doi.org/10.1016/S0377-2217\(03\)00069-9](https://doi.org/10.1016/S0377-2217(03)00069-9). URL: <http://www.sciencedirect.com/science/article/pii/S0377221703000699>.
- [44] Davi Vieira, Gabriel Gimenez, Guilherme Marmerola, and Vitor Estima. *XGBoost Survival Embeddings: improving statistical properties of XGBoost survival analysis implementation*. Version 0.2.3. 2021. URL: <http://github.com/loft-br/xgboost-survival-embeddings>.

- [45] Antonio Viganò, Marlene Dorgan, Jeanette Buckingham, Eduardo Bruera, and Maria E Suarez-Almazor. “Survival prediction in terminal cancer patients: a systematic review of the medical literature”. In: *Palliative Medicine* 14.5 (2000). PMID: 11064783, pp. 363–374. DOI: 10.1191/026921600701536192. eprint: <https://doi.org/10.1191/026921600701536192>. URL: <https://doi.org/10.1191/026921600701536192>.
- [46] Ping Wang, Yan Li, and Chandan K. Reddy. “Machine Learning for Survival Analysis: a Survey”. In: *ACM Computing Surveys* 51.6 (Feb. 2019). ISSN: 0360-0300. DOI: 10.1145/3214306. URL: <https://doi.org/10.1145/3214306>.
- [47] Lee-Jen Wei. “The Accelerated Failure Time model: a useful alternative to the Cox regression model in survival analysis”. In: *Statistics in Medicine* 11.14-15 (1992), 1871—1879. ISSN: 0277-6715. DOI: 10.1002/sim.4780111409. URL: <https://doi.org/10.1002/sim.4780111409>.
- [48] Tong Zhang and Bin Yu. “On the Convergence of Boosting Procedures”. In: *Proceedings of the Twentieth International Conference on International Conference on Machine Learning*. ICML’03. Washington, DC, USA: AAAI Press, 2003, 904–911. ISBN: 1577351894.

Appendices

A Code

The latest XGBoost package contains support for the AFT model. The source code is available at <https://github.com/dmlc/xgboost>.

The experiment code is available at <https://github.com/hcho3/XGBoostAFTPaperCode>.

B Tutorial

The first step is to express the labels in the form of a range, so that every data point has two numbers associated with it, namely the lower and upper bounds for the label. For uncensored labels, use a degenerate interval of form $[a, a]$.

B.1 Using XGBoost AFT in Python

Collect the lower bound numbers in one array (let's call it `y_lower`) and the upper bound number in another array (call it `y_upper`). Then pass the two arrays to the `xgboost.DMatrix` constructor via arguments `label_lower_bound` and `label_upper_bound`:

```
import numpy as np
import xgboost as xgb

# 4-by-2 Data matrix
X = np.array([[1, -1], [-1, 1], [0, 1], [1, 0]])

# Associate ranged labels with the data matrix.
# This example shows each kind of censored labels.
#

|                                   | <i>uncensored</i> | <i>right</i>          | <i>left</i>       | <i>interval</i>   |
|-----------------------------------|-------------------|-----------------------|-------------------|-------------------|
| <code>y_lower = np.array([</code> | <code>2.0,</code> | <code>3.0,</code>     | <code>0.0,</code> | <code>4.0]</code> |
| <code>y_upper = np.array([</code> | <code>2.0,</code> | <code>+np.inf,</code> | <code>4.0,</code> | <code>5.0]</code> |

dtrain = xgb.DMatrix(X, label_lower_bound=y_lower, label_upper_bound=y_upper)
```

Now we are ready to invoke the training API:

```
params = {'objective': 'survival:aft', 'eval_metric': 'aft-nloglik',
          'aft_loss_distribution': 'normal',
          'aft_loss_distribution_scale': 1.20,
```

```

      'tree_method': 'hist', 'learning_rate': 0.05, 'max_depth': 2}
bst = xgb.train(params, dtrain, num_boost_round=5, evals=[(dtrain, 'train')])

```

B.2 Using XGBoost AFT in R

Collect the lower bound numbers in one array (let's call it `y_lower`) and the upper bound number in another array (call it `y_upper`). Then associate the two arrays with a data matrix via calls to `setinfo`:

```

library(xgboost)

# 4-by-2 Data matrix
X <- matrix(c(1., -1., -1., 1., 0., 1., 1., 0.), nrow=4, ncol=2, byrow=TRUE)
dtrain <- xgb.DMatrix(X)

# Associate ranged labels with the data matrix.
# This example shows each kind of censored labels.
#           uncensored   right   left   interval
y_lower <- c(          2.,    3.,    0.,          4.)
y_upper <- c(          2.,  +Inf,    4.,          5.)
setinfo(dtrain, 'label_lower_bound', y_lower)
setinfo(dtrain, 'label_upper_bound', y_upper)

```

Now we are ready to invoke the training API:

```

params <- list(objective='survival:aft', eval_metric='aft-nloglik',
               aft_loss_distribution='normal',
               aft_loss_distribution_scale=1.20,
               tree_method='hist', learning_rate=0.05, max_depth=2)
watchlist <- list(train = dtrain)
bst <- xgb.train(params, dtrain, nrounds=5, watchlist)

```

B.3 Note on hyperparameters

Set objective hyperparameter to `survival:aft` and `eval_metric` to `aft-nloglik` in order to use the AFT model in XGBoost. The hyperparameter `aft_loss_distribution` corresponds to the distribution of Z in the AFT model, and `aft_loss_distribution_scale` corresponds to the scaling factor σ . Currently, you can choose from three probability distributions for `aft_loss_distribution`: `normal`, `logistic`, and `extreme`.

C Full proof for Definition 3, the gradient and hessian of the AFT loss

We first derive the first- and second-order partial derivatives of ℓ_{AFT} for the uncensored case, using basic rules of Calculus:

$$\ell(y, u) = -\ln \left[f_Z(s(y)) \cdot \frac{1}{\sigma y} \right] \quad (\text{C.1})$$

$$\frac{\partial \ell}{\partial u} = -\frac{\partial}{\partial u} \ln \left[f_Z(s(y)) \cdot \frac{1}{\sigma y} \right] \quad (\text{C.2})$$

$$= -\frac{1}{f_Z(s(y)) \cdot \frac{1}{\sigma y}} \cdot \frac{\partial}{\partial u} \left[f_Z(s(y)) \cdot \frac{1}{\sigma y} \right] \quad \text{Chain Rule} \quad (\text{C.3})$$

$$= -\frac{\sigma y}{f_Z(s(y))} \cdot \left[f'_Z(s(y)) \cdot \frac{\partial}{\partial u} \frac{\ln y - u}{\sigma} \cdot \frac{1}{\sigma y} \right] \quad \text{Chain Rule} \quad (\text{C.4})$$

$$= -\frac{\sigma y}{f_Z(s(y))} \cdot \left[f'_Z(s(y)) \cdot -\frac{1}{\sigma} \cdot \frac{1}{\sigma y} \right] \quad (\text{C.5})$$

$$= \frac{f'_Z(s(y))}{\sigma f_Z(s(y))} \quad (\text{C.6})$$

$$\frac{\partial^2 \ell}{\partial u^2} = \frac{\partial}{\partial u} \frac{\partial \ell}{\partial u} \quad (\text{C.7})$$

$$= \frac{\partial}{\partial u} \frac{f'_Z(s(y))}{\sigma f_Z(s(y))} \quad (\text{C.8})$$

$$= \frac{\partial / \partial u [f'_Z(s(y))] \cdot \sigma f_Z(s(y)) - f'_Z(s(y)) \cdot \sigma \partial / \partial u [f_Z(s(y))]}{\sigma^2 f_Z(s(y))^2} \quad \text{Quotient Rule} \quad (\text{C.9})$$

$$= \frac{f''_Z(s(y)) \cdot (-1/\sigma) \cdot \sigma f_Z(s(y)) - f'_Z(s(y)) \cdot \sigma f'_Z(s(y)) \cdot (-1/\sigma)}{\sigma^2 f_Z(s(y))^2} \quad \text{Chain Rule} \quad (\text{C.10})$$

$$= -\frac{f''_Z(s(y)) f_Z(s(y)) - f'_Z(s(y))^2}{\sigma^2 f_Z(s(y))^2} \quad (\text{C.11})$$

The censored case proceeds similarly:

$$\ell(y, u) = -\ln [F_Z(s(\bar{y})) - F_Z(s(\underline{y}))] \quad (\text{C.12})$$

$$\frac{\partial \ell}{\partial u} = -\frac{\partial}{\partial u} \ln [F_Z(s(\bar{y})) - F_Z(s(\underline{y}))] \quad (\text{C.13})$$

$$= -\frac{1}{F_Z(s(\bar{y})) - F_Z(s(\underline{y}))} \cdot \frac{\partial}{\partial u} [F_Z(s(\bar{y})) - F_Z(s(\underline{y}))] \quad \text{Chain Rule} \quad (\text{C.14})$$

$$= -\frac{1}{F_Z(s(\bar{y})) - F_Z(s(\underline{y}))} \cdot \left[f_Z(s(\bar{y})) \cdot -\frac{1}{\sigma} - f_Z(s(\underline{y})) \cdot -\frac{1}{\sigma} \right] \quad \text{Chain Rule; } f_Z = F'_Z \quad (\text{C.15})$$

$$= \frac{f_Z(s(\bar{y})) - f_Z(s(\underline{y}))}{\sigma[F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]} \quad (\text{C.16})$$

$$\frac{\partial^2 \ell}{\partial u^2} = \frac{\partial}{\partial u} \frac{\partial \ell}{\partial u} \quad (\text{C.17})$$

$$= \frac{\partial}{\partial u} \frac{f_Z(s(\bar{y})) - f_Z(s(\underline{y}))}{\sigma[F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]} \quad (\text{C.18})$$

$$= \frac{\partial / \partial u [f_Z(s(\bar{y})) - f_Z(s(\underline{y}))] \cdot \sigma[F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]}{\sigma^2[F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]^2} - \frac{[f_Z(s(\bar{y})) - f_Z(s(\underline{y}))] \cdot \sigma \partial / \partial u [F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]}{\sigma^2[F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]^2} \quad \text{Quotient Rule} \quad (\text{C.19})$$

$$= \frac{[f'_Z(s(\bar{y})) - f'_Z(s(\underline{y}))] \cdot (-1/\sigma) \cdot \sigma[F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]}{\sigma^2[F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]^2} - \frac{[f_Z(s(\bar{y})) - f_Z(s(\underline{y}))] \cdot \sigma[f_Z(s(\bar{y})) - f_Z(s(\underline{y}))] \cdot (-1/\sigma)}{\sigma^2[F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]^2} \quad \text{Chain Rule; } f_Z = F'_Z \quad (\text{C.20})$$

$$= \frac{-[F_Z(s(\bar{y})) - F_Z(s(\underline{y}))][f'_Z(s(\bar{y})) - f'_Z(s(\underline{y}))] + [f_Z(s(\bar{y})) - f_Z(s(\underline{y}))]^2}{\sigma^2[F_Z(s(\bar{y})) - F_Z(s(\underline{y}))]^2} \quad (\text{C.21})$$

D Effect of hyperparameters on model performance

The goal of the experiment is to quantify the impact of the hyperparameter choice on the generalization performance. For each model produced by each hyperparameter combination, we compute the test accuracy using the held-out test set.

The following tables show the validation and test accuracy of all models produced by the hyperparameter search. See Section 3.3 for detailed instructions.

Is the AFT method very sensitive to the hyperparameter choice? If yes, then we will need to run many trials of hyperparameter search and the performance benefit of XGBoost will

be negated. If not, then we will be able to run a comparatively small number of trials and we will be able to obtain an optimal model in short amount of time.

The answer to the question of the preceding paragraph is yes. For all data sets, running 1000 trials of hyperparameter search did not produce significant advantage over running only 100 trials, when measured in the test accuracy measure. Thus, we can simply run 100 rounds of hyperparameter search and obtain a close-to-optimal model, at least when measured in the aggregate with the test accuracy.

Unfortunately, this analysis has a gaping hole: a model being optimal in the aggregate does not mean it is well calibrated in individual predictions. Two models may have similar test accuracy but produce wildly differing prediction for some inputs. See discussion in Section 4.

Table D.1: Comparison of hyperparameter search methods. Both validation and test accuracy are averages over the 4 outer cross-validation folds.

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
Dataset ATAC_JV_adipose, normal distribution					
0	Grid	max_depth, reg_lambda	54	0.715249	0.448685
1	Grid	learning_rate, max_depth	36	0.710944	0.446252
2	Grid	reg_alpha	6	0.686967	0.445603
3	Grid	max_depth, min_child_weight	45	0.708613	0.443379
4	Random	All six	1000	0.724057	0.441571
5	Grid	max_depth	9	0.704368	0.440111
6	Baseline	None (all defaults)	1	0.677217	0.435522
7	Random	All six	100	0.718862	0.434572
8	Grid	min_child_weight	5	0.684872	0.432649
9	Grid	max_depth, reg_alpha	54	0.710192	0.427622
10	Grid	reg_lambda	6	0.698796	0.425557
11	Grid	max_depth, aft_loss_distribution_scale	54	0.719006	0.425557
12	Grid	learning_rate, reg_alpha	24	0.704978	0.423541
13	Grid	reg_alpha, aft_loss_distribution_scale	36	0.709728	0.420552
14	Grid	min_child_weight, reg_alpha	30	0.694428	0.420089
15	Grid	reg_lambda, aft_loss_distribution_scale	36	0.709096	0.418952
16	Grid	learning_rate, min_child_weight	20	0.709576	0.418629
17	Grid	reg_alpha, reg_lambda	36	0.700208	0.416635
18	Grid	learning_rate, aft_loss_distribution_scale	24	0.707424	0.415268
19	Grid	learning_rate, reg_lambda	24	0.708276	0.413947
20	Grid	aft_loss_distribution_scale	6	0.700577	0.412047
21	Grid	learning_rate	4	0.698327	0.411630
22	Grid	min_child_weight, reg_lambda	30	0.703793	0.400971
23	Grid	min_child_weight, aft_loss_distribution_scale	30	0.706980	0.395038
Dataset ATAC_JV_adipose, logistic distribution					
0	Grid	learning_rate, max_depth	36	0.717829	0.451651
1	Grid	max_depth, aft_loss_distribution_scale	54	0.718995	0.445811
2	Grid	aft_loss_distribution_scale	6	0.705980	0.439856
3	Grid	reg_alpha, aft_loss_distribution_scale	36	0.711729	0.439092
4	Grid	min_child_weight, aft_loss_distribution_scale	30	0.706704	0.437747

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
5	Grid	learning_rate	4	0.707096	0.430030
6	Random	All six	100	0.720789	0.426439
7	Grid	reg_lambda	6	0.704542	0.425743
8	Grid	learning_rate, min_child_weight	20	0.709754	0.424283
9	Grid	min_child_weight, reg_lambda	30	0.707443	0.415964
10	Grid	learning_rate, reg_lambda	24	0.712631	0.415662
11	Grid	learning_rate, reg_alpha	24	0.708575	0.409893
12	Grid	reg_lambda, aft_loss_distribution_scale	36	0.716163	0.407899
13	Grid	learning_rate, aft_loss_distribution_scale	24	0.711026	0.407691
14	Random	All six	1000	0.727451	0.406277
15	Grid	max_depth, reg_lambda	54	0.716056	0.406069
16	Grid	reg_alpha, reg_lambda	36	0.710365	0.405976
17	Grid	min_child_weight, reg_alpha	30	0.437865	0.016986
18	Grid	reg_alpha	6	0.437119	0.015943
19	Grid	max_depth, reg_alpha	54	0.437119	0.015943
20	Baseline	None (all defaults)	1	0.021481	0.000000
21	Grid	max_depth	9	0.021481	0.000000
22	Grid	min_child_weight	5	0.021481	0.000000
23	Grid	max_depth, min_child_weight	45	0.021481	0.000000
Dataset ATAC_JV_adipose, extreme distribution					
0	Grid	learning_rate, max_depth	36	0.717760	0.451024
1	Grid	max_depth, aft_loss_distribution_scale	54	0.716240	0.450167
2	Grid	max_depth, reg_lambda	54	0.717171	0.444699
3	Grid	max_depth	9	0.713462	0.440481
4	Grid	reg_alpha, aft_loss_distribution_scale	36	0.711823	0.435939
5	Random	All six	1000	0.724805	0.430863
6	Grid	aft_loss_distribution_scale	6	0.702618	0.430748
7	Grid	learning_rate	4	0.706831	0.423656
8	Grid	learning_rate, aft_loss_distribution_scale	24	0.712637	0.423587
9	Grid	min_child_weight	5	0.704203	0.423078
10	Random	All six	100	0.721105	0.422962
11	Baseline	None (all defaults)	1	0.699781	0.422798
12	Grid	max_depth, min_child_weight	45	0.718588	0.422521
13	Grid	learning_rate, min_child_weight	20	0.712486	0.420204
14	Grid	learning_rate, reg_lambda	24	0.712615	0.419810
15	Grid	max_depth, reg_alpha	54	0.719171	0.419763
16	Grid	reg_alpha, reg_lambda	36	0.708990	0.417446
17	Grid	min_child_weight, reg_alpha	30	0.717606	0.414851
18	Grid	reg_lambda	6	0.706155	0.413622
19	Grid	min_child_weight, aft_loss_distribution_scale	30	0.712047	0.410332
20	Grid	reg_alpha	6	0.706762	0.409381
21	Grid	reg_lambda, aft_loss_distribution_scale	36	0.714897	0.408896
22	Grid	min_child_weight, reg_lambda	30	0.711983	0.400645
23	Grid	learning_rate, reg_alpha	24	0.711072	0.397679
Dataset CTCF_TDH_ENCODE, normal distribution					
0	Grid	max_depth, aft_loss_distribution_scale	54	0.882398	0.807692
1	Grid	learning_rate, aft_loss_distribution_scale	24	0.867323	0.798077
2	Grid	min_child_weight, aft_loss_distribution_scale	30	0.863787	0.798077
3	Random	All six	1000	0.884321	0.788462

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
4	Grid	learning_rate, reg_lambda	24	0.854172	0.788462
5	Grid	aft_loss_distribution_scale	6	0.861864	0.788462
6	Grid	learning_rate, max_depth	36	0.875066	0.788462
7	Grid	reg_alpha, aft_loss_distribution_scale	36	0.868936	0.788462
8	Grid	learning_rate, reg_alpha	24	0.858018	0.783654
9	Grid	max_depth, reg_alpha	54	0.859049	0.783654
10	Random	All six	100	0.876939	0.778846
11	Grid	reg_lambda, aft_loss_distribution_scale	36	0.874965	0.774038
12	Grid	learning_rate	4	0.840710	0.759615
13	Grid	reg_alpha, reg_lambda	36	0.847821	0.759615
14	Grid	learning_rate, min_child_weight	20	0.850326	0.754808
15	Grid	max_depth, reg_lambda	54	0.857487	0.730769
16	Grid	max_depth	9	0.845948	0.721154
17	Grid	min_child_weight, reg_lambda	30	0.842412	0.721154
18	Grid	max_depth, min_child_weight	45	0.852710	0.697115
19	Grid	reg_lambda	6	0.840439	0.692308
20	Grid	min_child_weight, reg_alpha	30	0.836062	0.692308
21	Baseline	None (all defaults)	1	0.826446	0.682692
22	Grid	min_child_weight	5	0.826446	0.677885
23	Grid	reg_alpha	6	0.828369	0.673077
Dataset CTCF_TDH_ENCODE, logistic distribution					
0	Grid	min_child_weight, aft_loss_distribution_scale	30	0.871530	0.831731
1	Random	All six	100	0.884321	0.826923
2	Grid	max_depth, aft_loss_distribution_scale	54	0.884321	0.826923
3	Grid	learning_rate, reg_alpha	24	0.869246	0.822115
4	Grid	learning_rate	4	0.865400	0.817308
5	Grid	max_depth	9	0.877299	0.817308
6	Grid	max_depth, reg_alpha	54	0.880785	0.812500
7	Grid	reg_alpha	6	0.860561	0.807692
8	Grid	reg_alpha, aft_loss_distribution_scale	36	0.874345	0.807692
9	Grid	learning_rate, max_depth	36	0.880475	0.802885
10	Grid	learning_rate, reg_lambda	24	0.869246	0.802885
11	Baseline	None (all defaults)	1	0.857076	0.798077
12	Random	All six	1000	0.889780	0.798077
13	Grid	reg_alpha, reg_lambda	36	0.863838	0.798077
14	Grid	aft_loss_distribution_scale	6	0.867684	0.798077
15	Grid	max_depth, min_child_weight	45	0.884321	0.793269
16	Grid	max_depth, reg_lambda	54	0.878862	0.793269
17	Grid	reg_lambda	6	0.861864	0.788462
18	Grid	min_child_weight, reg_lambda	30	0.865710	0.788462
19	Grid	reg_lambda, aft_loss_distribution_scale	36	0.873092	0.783654
20	Grid	learning_rate, min_child_weight	20	0.869246	0.783654
21	Grid	min_child_weight	5	0.862485	0.759615
22	Grid	learning_rate, aft_loss_distribution_scale	24	0.875016	0.750000
23	Grid	min_child_weight, reg_alpha	30	0.869556	0.725962
Dataset CTCF_TDH_ENCODE, extreme distribution					
0	Grid	max_depth, reg_alpha	54	0.850996	0.798077
1	Grid	max_depth	9	0.845227	0.788462
2	Grid	learning_rate	4	0.841381	0.783654

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
3	Grid	learning_rate, reg_lambda	24	0.845227	0.783654
4	Grid	learning_rate, max_depth	36	0.847201	0.778846
5	Grid	learning_rate, aft_loss_distribution_scale	24	0.848763	0.778846
6	Random	All six	1000	0.870859	0.774038
7	Grid	learning_rate, min_child_weight	20	0.850996	0.774038
8	Grid	learning_rate, reg_alpha	24	0.850996	0.769231
9	Grid	reg_alpha, reg_lambda	36	0.852249	0.769231
10	Random	All six	100	0.854843	0.764423
11	Grid	max_depth, min_child_weight	45	0.858278	0.759615
12	Grid	min_child_weight	5	0.847150	0.754808
13	Baseline	None (all defaults)	1	0.835612	0.750000
14	Grid	reg_alpha	6	0.847150	0.745192
15	Grid	min_child_weight, reg_alpha	30	0.863787	0.745192
16	Grid	max_depth, aft_loss_distribution_scale	54	0.861915	0.740385
17	Grid	reg_alpha, aft_loss_distribution_scale	36	0.856044	0.740385
18	Grid	reg_lambda, aft_loss_distribution_scale	36	0.854532	0.740385
19	Grid	reg_lambda	6	0.843304	0.735577
20	Grid	min_child_weight, reg_lambda	30	0.854842	0.735577
21	Grid	aft_loss_distribution_scale	6	0.841381	0.706731
22	Grid	max_depth, reg_lambda	54	0.852919	0.706731
23	Grid	min_child_weight, aft_loss_distribution_scale	30	0.854843	0.682692
Dataset H3K27ac-H3K4me3_TDHAM_BP, normal distribution					
0	Grid	max_depth, aft_loss_distribution_scale	54	0.884932	0.738933
1	Grid	learning_rate, min_child_weight	20	0.881360	0.726499
2	Grid	reg_alpha, aft_loss_distribution_scale	36	0.883761	0.725599
3	Grid	max_depth, reg_alpha	54	0.879121	0.720361
4	Random	All six	100	0.885043	0.719415
5	Grid	learning_rate, aft_loss_distribution_scale	24	0.883147	0.718623
6	Grid	learning_rate	4	0.880459	0.718509
7	Grid	learning_rate, reg_alpha	24	0.883937	0.718318
8	Grid	min_child_weight, reg_lambda	30	0.879320	0.717944
9	Grid	learning_rate, max_depth	36	0.885600	0.716705
10	Grid	learning_rate, reg_lambda	24	0.883249	0.715441
11	Grid	max_depth	9	0.877523	0.714167
12	Random	All six	1000	0.889028	0.713855
13	Grid	max_depth, reg_lambda	54	0.880891	0.713815
14	Grid	max_depth, min_child_weight	45	0.879840	0.711777
15	Grid	reg_lambda, aft_loss_distribution_scale	36	0.884314	0.710863
16	Grid	aft_loss_distribution_scale	6	0.881473	0.710385
17	Grid	reg_alpha	6	0.875224	0.709079
18	Grid	min_child_weight, aft_loss_distribution_scale	30	0.881970	0.708373
19	Grid	reg_alpha, reg_lambda	36	0.878228	0.707810
20	Grid	min_child_weight, reg_alpha	30	0.876971	0.707595
21	Grid	reg_lambda	6	0.876496	0.703961
22	Baseline	None (all defaults)	1	0.874224	0.702230
23	Grid	min_child_weight	5	0.875278	0.692765
Dataset H3K27ac-H3K4me3_TDHAM_BP, logistic distribution					
0	Grid	max_depth, aft_loss_distribution_scale	54	0.883494	0.736861
1	Grid	learning_rate, reg_alpha	24	0.881191	0.734715

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
2	Grid	max_depth, reg_lambda	54	0.883060	0.729967
3	Grid	min_child_weight, reg_lambda	30	0.881664	0.726794
4	Grid	reg_lambda	6	0.880549	0.724545
5	Grid	reg_alpha, aft_loss_distribution_scale	36	0.881595	0.722549
6	Grid	learning_rate, reg_lambda	24	0.883224	0.722238
7	Grid	learning_rate, min_child_weight	20	0.880681	0.720969
8	Grid	learning_rate, aft_loss_distribution_scale	24	0.883757	0.719983
9	Grid	learning_rate	4	0.879262	0.718029
10	Random	All six	1000	0.887514	0.717477
11	Grid	aft_loss_distribution_scale	6	0.880184	0.717038
12	Grid	reg_alpha, reg_lambda	36	0.881584	0.716748
13	Random	All six	100	0.883953	0.716571
14	Grid	learning_rate, max_depth	36	0.883329	0.716231
15	Grid	reg_lambda, aft_loss_distribution_scale	36	0.883250	0.715480
16	Grid	max_depth, reg_alpha	54	0.837738	0.714114
17	Grid	max_depth, min_child_weight	45	0.818940	0.713289
18	Grid	reg_alpha	6	0.835391	0.713041
19	Grid	min_child_weight, aft_loss_distribution_scale	30	0.882020	0.711488
20	Grid	min_child_weight, reg_alpha	30	0.837452	0.708518
21	Grid	min_child_weight	5	0.817225	0.707281
22	Baseline	None (all defaults)	1	0.812914	0.698842
23	Grid	max_depth	9	0.816306	0.686587
Dataset H3K27ac-H3K4me3_TDHAM_BP, extreme distribution					
0	Grid	max_depth, reg_lambda	54	0.878618	0.724906
1	Grid	min_child_weight, reg_alpha	30	0.876197	0.723198
2	Random	All six	1000	0.881604	0.717300
3	Grid	max_depth, reg_alpha	54	0.878082	0.715187
4	Grid	learning_rate, reg_lambda	24	0.877669	0.711786
5	Grid	min_child_weight	5	0.873195	0.711241
6	Grid	reg_alpha, reg_lambda	36	0.877408	0.710815
7	Grid	reg_alpha	6	0.874107	0.709582
8	Grid	min_child_weight, reg_lambda	30	0.876320	0.708710
9	Grid	min_child_weight, aft_loss_distribution_scale	30	0.880395	0.706504
10	Baseline	None (all defaults)	1	0.869358	0.705427
11	Grid	reg_lambda, aft_loss_distribution_scale	36	0.880276	0.704131
12	Grid	learning_rate, max_depth	36	0.877551	0.703992
13	Grid	learning_rate, min_child_weight	20	0.877098	0.703003
14	Grid	reg_lambda	6	0.873835	0.702990
15	Grid	max_depth, aft_loss_distribution_scale	54	0.880390	0.702945
16	Grid	learning_rate	4	0.875194	0.701294
17	Grid	max_depth	9	0.876619	0.701220
18	Grid	aft_loss_distribution_scale	6	0.879035	0.701024
19	Grid	learning_rate, aft_loss_distribution_scale	24	0.879532	0.700521
20	Random	All six	100	0.879358	0.698957
21	Grid	reg_alpha, aft_loss_distribution_scale	36	0.879035	0.698509
22	Grid	max_depth, min_child_weight	45	0.876620	0.698202
23	Grid	learning_rate, reg_alpha	24	0.875916	0.697114
Dataset H3K27ac_TDH_some, normal distribution					
0	Grid	min_child_weight	5	0.544659	0.355263

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
1	Grid	min_child_weight, reg_alpha	30	0.563655	0.322368
2	Grid	reg_alpha	6	0.548655	0.315789
3	Grid	reg_alpha, reg_lambda	36	0.570076	0.315789
4	Random	All six	1000	0.609261	0.309211
5	Grid	reg_lambda	6	0.556364	0.309211
6	Grid	learning_rate, reg_alpha	24	0.565663	0.309211
7	Grid	min_child_weight, reg_lambda	30	0.574413	0.309211
8	Grid	max_depth	9	0.548447	0.302632
9	Grid	max_depth, reg_alpha	54	0.567955	0.302632
10	Grid	min_child_weight, aft_loss_distribution_scale	30	0.570795	0.302632
11	Grid	learning_rate, min_child_weight	20	0.557746	0.296053
12	Grid	reg_lambda, aft_loss_distribution_scale	36	0.587462	0.296053
13	Grid	max_depth, min_child_weight	45	0.565947	0.289474
14	Grid	reg_alpha, aft_loss_distribution_scale	36	0.572708	0.289474
15	Baseline	None (all defaults)	1	0.522481	0.282895
16	Grid	learning_rate	4	0.540284	0.282895
17	Grid	learning_rate, max_depth	36	0.570530	0.276316
18	Random	All six	100	0.598258	0.250000
19	Grid	aft_loss_distribution_scale	6	0.559167	0.250000
20	Grid	learning_rate, aft_loss_distribution_scale	24	0.565795	0.243421
21	Grid	learning_rate, reg_lambda	24	0.586174	0.236842
22	Grid	max_depth, aft_loss_distribution_scale	54	0.584716	0.230263
23	Grid	max_depth, reg_lambda	54	0.568201	0.217105
Dataset H3K27ac_TDH_some, logistic distribution					
0	Grid	min_child_weight, reg_lambda	30	0.564413	0.375000
1	Grid	reg_lambda, aft_loss_distribution_scale	36	0.585076	0.375000
2	Grid	min_child_weight, reg_alpha	30	0.562708	0.361842
3	Grid	min_child_weight	5	0.558163	0.348684
4	Grid	reg_lambda	6	0.556913	0.335526
5	Grid	learning_rate, min_child_weight	20	0.566042	0.328947
6	Random	All six	100	0.597633	0.309211
7	Grid	learning_rate	4	0.562292	0.302632
8	Grid	reg_alpha	6	0.555208	0.302632
9	Grid	learning_rate, reg_alpha	24	0.562292	0.302632
10	Random	All six	1000	0.601174	0.296053
11	Grid	max_depth	9	0.562083	0.289474
12	Grid	aft_loss_distribution_scale	6	0.562121	0.289474
13	Grid	max_depth, reg_alpha	54	0.565000	0.289474
14	Grid	min_child_weight, aft_loss_distribution_scale	30	0.576458	0.289474
15	Grid	reg_alpha, reg_lambda	36	0.570417	0.289474
16	Grid	reg_alpha, aft_loss_distribution_scale	36	0.577955	0.289474
17	Grid	learning_rate, aft_loss_distribution_scale	24	0.574621	0.282895
18	Grid	max_depth, min_child_weight	45	0.579205	0.282895
19	Grid	learning_rate, reg_lambda	24	0.576875	0.276316
20	Grid	max_depth, reg_lambda	54	0.572083	0.276316
21	Baseline	None (all defaults)	1	0.550663	0.269737
22	Grid	learning_rate, max_depth	36	0.569583	0.256579
23	Grid	max_depth, aft_loss_distribution_scale	54	0.578750	0.250000
Dataset H3K27ac_TDH_some, extreme distribution					

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
0	Grid	max_depth, reg_alpha	54	0.569413	0.440789
1	Grid	max_depth	9	0.555208	0.427632
2	Grid	max_depth, min_child_weight	45	0.564792	0.427632
3	Grid	min_child_weight	5	0.553788	0.421053
4	Grid	learning_rate, max_depth	36	0.570208	0.407895
5	Grid	min_child_weight, reg_alpha	30	0.566042	0.401316
6	Grid	learning_rate, min_child_weight	20	0.570208	0.368421
7	Grid	min_child_weight, aft_loss_distribution_scale	30	0.583087	0.368421
8	Baseline	None (all defaults)	1	0.526913	0.355263
9	Grid	learning_rate, reg_alpha	24	0.566667	0.355263
10	Grid	reg_lambda, aft_loss_distribution_scale	36	0.602670	0.355263
11	Grid	reg_alpha	6	0.555455	0.348684
12	Grid	learning_rate	4	0.560208	0.342105
13	Grid	reg_lambda	6	0.569792	0.342105
14	Grid	max_depth, aft_loss_distribution_scale	54	0.586629	0.328947
15	Grid	learning_rate, reg_lambda	24	0.578958	0.315789
16	Random	All six	1000	0.617254	0.302632
17	Random	All six	100	0.613712	0.302632
18	Grid	max_depth, reg_lambda	54	0.598125	0.302632
19	Grid	reg_alpha, reg_lambda	36	0.593296	0.256579
20	Grid	aft_loss_distribution_scale	6	0.576212	0.250000
21	Grid	learning_rate, aft_loss_distribution_scale	24	0.576212	0.250000
22	Grid	min_child_weight, reg_lambda	30	0.597330	0.243421
23	Grid	reg_alpha, aft_loss_distribution_scale	36	0.589546	0.223684
Dataset H3K27me3_RL_cancer, normal distribution					
0	Grid	max_depth, reg_lambda	54	0.646010	0.444079
1	Random	All six	1000	0.675466	0.421053
2	Grid	reg_lambda, aft_loss_distribution_scale	36	0.646969	0.388158
3	Random	All six	100	0.655305	0.375000
4	Grid	min_child_weight, reg_lambda	30	0.623847	0.375000
5	Grid	min_child_weight	5	0.615952	0.358553
6	Grid	learning_rate, reg_alpha	24	0.627534	0.355263
7	Grid	min_child_weight, reg_alpha	30	0.633020	0.351974
8	Grid	learning_rate, reg_lambda	24	0.639905	0.351974
9	Grid	reg_lambda	6	0.614571	0.348684
10	Grid	reg_alpha	6	0.622270	0.345395
11	Grid	learning_rate, min_child_weight	20	0.623352	0.332237
12	Grid	max_depth, aft_loss_distribution_scale	54	0.642759	0.332237
13	Baseline	None (all defaults)	1	0.609307	0.332237
14	Grid	min_child_weight, aft_loss_distribution_scale	30	0.634842	0.328947
15	Grid	learning_rate	4	0.620126	0.319079
16	Grid	max_depth, reg_alpha	54	0.644709	0.319079
17	Grid	reg_alpha, reg_lambda	36	0.631237	0.309211
18	Grid	max_depth	9	0.626917	0.305921
19	Grid	max_depth, min_child_weight	45	0.629549	0.305921
20	Grid	learning_rate, aft_loss_distribution_scale	24	0.631610	0.302632
21	Grid	reg_alpha, aft_loss_distribution_scale	36	0.629485	0.302632
22	Grid	learning_rate, max_depth	36	0.639296	0.296053
23	Grid	aft_loss_distribution_scale	6	0.623847	0.276316

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
Dataset H3K27me3_RL_cancer, logistic distribution					
0	Grid	aft_loss_distribution_scale	6	0.623945	0.437500
1	Grid	min_child_weight, aft_loss_distribution_scale	30	0.623945	0.437500
2	Grid	reg_alpha	6	0.615499	0.427632
3	Grid	learning_rate, reg_lambda	24	0.633127	0.424342
4	Grid	min_child_weight, reg_alpha	30	0.620984	0.421053
5	Grid	max_depth, aft_loss_distribution_scale	54	0.634811	0.411184
6	Grid	min_child_weight	5	0.614923	0.411184
7	Grid	learning_rate, reg_alpha	24	0.631806	0.401316
8	Grid	min_child_weight, reg_lambda	30	0.622968	0.398026
9	Grid	reg_alpha, aft_loss_distribution_scale	36	0.636061	0.394737
10	Baseline	None (all defaults)	1	0.609154	0.391447
11	Grid	reg_lambda	6	0.615561	0.384868
12	Grid	learning_rate, max_depth	36	0.646977	0.384868
13	Grid	reg_alpha, reg_lambda	36	0.624317	0.384868
14	Grid	max_depth	9	0.642827	0.384868
15	Grid	max_depth, min_child_weight	45	0.642827	0.384868
16	Grid	max_depth, reg_alpha	54	0.644679	0.384868
17	Grid	max_depth, reg_lambda	54	0.642827	0.384868
18	Random	All six	1000	0.659736	0.378289
19	Grid	learning_rate	4	0.625585	0.378289
20	Grid	learning_rate, min_child_weight	20	0.628217	0.371711
21	Grid	reg_lambda, aft_loss_distribution_scale	36	0.644768	0.371711
22	Random	All six	100	0.645898	0.358553
23	Grid	learning_rate, aft_loss_distribution_scale	24	0.642394	0.358553
Dataset H3K27me3_RL_cancer, extreme distribution					
0	Grid	learning_rate, aft_loss_distribution_scale	24	0.642351	0.460526
1	Grid	max_depth, aft_loss_distribution_scale	54	0.649722	0.460526
2	Grid	reg_alpha, reg_lambda	36	0.635641	0.460526
3	Grid	min_child_weight, reg_alpha	30	0.632692	0.453947
4	Baseline	None (all defaults)	1	0.606034	0.450658
5	Grid	reg_alpha	6	0.629199	0.447368
6	Grid	aft_loss_distribution_scale	6	0.629934	0.447368
7	Grid	min_child_weight, aft_loss_distribution_scale	30	0.639285	0.447368
8	Grid	reg_alpha, aft_loss_distribution_scale	36	0.647543	0.440789
9	Grid	reg_lambda	6	0.626211	0.434211
10	Grid	reg_lambda, aft_loss_distribution_scale	36	0.643589	0.430921
11	Grid	learning_rate	4	0.618405	0.421053
12	Grid	max_depth	9	0.630837	0.421053
13	Grid	min_child_weight	5	0.622225	0.414474
14	Random	All six	1000	0.653055	0.414474
15	Random	All six	100	0.644059	0.411184
16	Grid	max_depth, reg_alpha	54	0.637652	0.407895
17	Grid	max_depth, min_child_weight	45	0.634170	0.407895
18	Grid	max_depth, reg_lambda	54	0.637635	0.407895
19	Grid	learning_rate, max_depth	36	0.632575	0.401316
20	Grid	learning_rate, reg_alpha	24	0.631050	0.401316
21	Grid	learning_rate, min_child_weight	20	0.628988	0.394737
22	Grid	min_child_weight, reg_lambda	30	0.636605	0.384868

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
23	Grid	learning_rate, reg_lambda	24	0.628063	0.348684
Dataset H3K27me3_TDH_some, normal distribution					
0	Grid	max_depth, reg_alpha	54	0.572917	0.476190
1	Grid	max_depth, reg_lambda	54	0.581845	0.464286
2	Grid	learning_rate, reg_alpha	24	0.590774	0.455357
3	Random	All six	100	0.594643	0.434524
4	Grid	max_depth	9	0.547917	0.434524
5	Baseline	None (all defaults)	1	0.539583	0.413690
6	Grid	min_child_weight	5	0.539583	0.413690
7	Grid	reg_alpha	6	0.547917	0.413690
8	Grid	max_depth, min_child_weight	45	0.578869	0.413690
9	Random	All six	1000	0.633036	0.398810
10	Grid	learning_rate, max_depth	36	0.575000	0.398810
11	Grid	reg_alpha, reg_lambda	36	0.581845	0.398810
12	Grid	min_child_weight, reg_alpha	30	0.556250	0.392857
13	Grid	learning_rate, aft_loss_distribution_scale	24	0.613393	0.383929
14	Grid	reg_lambda, aft_loss_distribution_scale	36	0.587202	0.383929
15	Grid	learning_rate, reg_lambda	24	0.598512	0.377976
16	Grid	learning_rate	4	0.558631	0.372024
17	Grid	max_depth, aft_loss_distribution_scale	54	0.625595	0.363095
18	Grid	min_child_weight, aft_loss_distribution_scale	30	0.580060	0.363095
19	Grid	reg_alpha, aft_loss_distribution_scale	36	0.608631	0.363095
20	Grid	learning_rate, min_child_weight	20	0.565774	0.351190
21	Grid	reg_lambda	6	0.558036	0.336310
22	Grid	aft_loss_distribution_scale	6	0.578869	0.321429
23	Grid	min_child_weight, reg_lambda	30	0.575893	0.315476
Dataset H3K27me3_TDH_some, logistic distribution					
0	Grid	reg_alpha	6	0.555060	0.532738
1	Grid	reg_alpha, reg_lambda	36	0.564583	0.532738
2	Grid	min_child_weight, reg_alpha	30	0.557441	0.511905
3	Grid	min_child_weight	5	0.544941	0.482143
4	Grid	min_child_weight, reg_lambda	30	0.550893	0.461310
5	Grid	min_child_weight, aft_loss_distribution_scale	30	0.598810	0.446429
6	Grid	learning_rate, reg_lambda	24	0.579464	0.440476
7	Grid	max_depth, min_child_weight	45	0.578572	0.425595
8	Grid	reg_alpha, aft_loss_distribution_scale	36	0.594048	0.425595
9	Grid	learning_rate, min_child_weight	20	0.575893	0.419643
10	Grid	reg_lambda, aft_loss_distribution_scale	36	0.587798	0.419643
11	Grid	reg_lambda	6	0.550893	0.419643
12	Grid	learning_rate, max_depth	36	0.583036	0.419643
13	Grid	learning_rate, aft_loss_distribution_scale	24	0.588095	0.404762
14	Grid	max_depth, aft_loss_distribution_scale	54	0.588095	0.404762
15	Random	All six	1000	0.622321	0.401786
16	Baseline	None (all defaults)	1	0.541369	0.398810
17	Grid	max_depth	9	0.557143	0.398810
18	Grid	aft_loss_distribution_scale	6	0.571429	0.383929
19	Grid	learning_rate, reg_alpha	24	0.580952	0.383929
20	Grid	max_depth, reg_lambda	54	0.572619	0.383929
21	Grid	learning_rate	4	0.567560	0.377976

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
22	Grid	max_depth, reg_alpha	54	0.563095	0.342262
23	Random	All six	100	0.616964	0.321429
Dataset H3K27me3_TDH_some, extreme distribution					
0	Baseline	None (all defaults)	1	0.552083	0.523810
1	Grid	min_child_weight, reg_lambda	30	0.598512	0.523810
2	Grid	min_child_weight	5	0.582143	0.482143
3	Grid	learning_rate, min_child_weight	20	0.589286	0.482143
4	Grid	reg_lambda, aft_loss_distribution_scale	36	0.629464	0.476190
5	Grid	reg_lambda	6	0.591369	0.467262
6	Grid	max_depth	9	0.566369	0.461310
7	Grid	max_depth, min_child_weight	45	0.597619	0.461310
8	Grid	learning_rate, reg_alpha	24	0.583036	0.455357
9	Grid	min_child_weight, reg_alpha	30	0.582143	0.446429
10	Grid	reg_alpha, reg_lambda	36	0.604464	0.425595
11	Grid	min_child_weight, aft_loss_distribution_scale	30	0.609524	0.404762
12	Grid	reg_alpha	6	0.561607	0.383929
13	Grid	learning_rate, reg_lambda	24	0.612798	0.383929
14	Grid	learning_rate, aft_loss_distribution_scale	24	0.611012	0.383929
15	Grid	max_depth, reg_lambda	54	0.598512	0.363095
16	Random	All six	1000	0.665774	0.357143
17	Grid	aft_loss_distribution_scale	6	0.596429	0.342262
18	Grid	reg_alpha, aft_loss_distribution_scale	36	0.597322	0.342262
19	Grid	max_depth, aft_loss_distribution_scale	54	0.634524	0.327381
20	Grid	max_depth, reg_alpha	54	0.588988	0.321429
21	Grid	learning_rate	4	0.572619	0.321429
22	Grid	learning_rate, max_depth	36	0.596131	0.294643
23	Random	All six	100	0.602381	0.258929
Dataset H3K36me3_AM_immune, normal distribution					
0	Grid	min_child_weight, reg_lambda	30	0.939712	0.881349
1	Random	All six	100	0.945227	0.872817
2	Grid	max_depth, reg_alpha	54	0.942738	0.872222
3	Grid	learning_rate, max_depth	36	0.940332	0.867659
4	Grid	learning_rate, min_child_weight	20	0.940506	0.864683
5	Grid	aft_loss_distribution_scale	6	0.938072	0.863889
6	Grid	min_child_weight, aft_loss_distribution_scale	30	0.938818	0.863889
7	Grid	reg_alpha, aft_loss_distribution_scale	36	0.938072	0.863889
8	Grid	max_depth	9	0.939503	0.863095
9	Grid	learning_rate, aft_loss_distribution_scale	24	0.940347	0.860714
10	Grid	learning_rate, reg_alpha	24	0.938024	0.859722
11	Random	All six	1000	0.948306	0.859722
12	Grid	reg_lambda, aft_loss_distribution_scale	36	0.940513	0.858135
13	Grid	max_depth, reg_lambda	54	0.941155	0.857937
14	Baseline	None (all defaults)	1	0.923607	0.856548
15	Grid	max_depth, min_child_weight	45	0.942839	0.850992
16	Grid	reg_alpha	6	0.934849	0.850198
17	Grid	reg_lambda	6	0.933234	0.844246
18	Grid	reg_alpha, reg_lambda	36	0.938024	0.840675
19	Grid	max_depth, aft_loss_distribution_scale	54	0.943565	0.839683
20	Grid	learning_rate	4	0.930826	0.838492

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
21	Grid	min_child_weight	5	0.935744	0.828968
22	Grid	min_child_weight, reg_alpha	30	0.938077	0.822421
23	Grid	learning_rate, reg_lambda	24	0.938822	0.822024
Dataset H3K36me3_AM_immune, logistic distribution					
0	Random	All six	1000	0.945865	0.909921
1	Grid	reg_lambda, aft_loss_distribution_scale	36	0.938072	0.890873
2	Grid	reg_lambda	6	0.935093	0.875397
3	Grid	reg_alpha, reg_lambda	36	0.935839	0.872421
4	Grid	max_depth, reg_lambda	54	0.935839	0.868254
5	Grid	learning_rate	4	0.932492	0.867262
6	Grid	learning_rate, reg_alpha	24	0.932492	0.867262
7	Grid	learning_rate, min_child_weight	20	0.932503	0.866667
8	Grid	learning_rate, max_depth	36	0.936521	0.856349
9	Grid	min_child_weight, aft_loss_distribution_scale	30	0.934246	0.851786
10	Random	All six	100	0.939702	0.819444
11	Grid	min_child_weight, reg_lambda	30	0.935839	0.655159
12	Grid	aft_loss_distribution_scale	6	0.933387	0.653373
13	Grid	reg_alpha, aft_loss_distribution_scale	36	0.934235	0.651389
14	Grid	learning_rate, aft_loss_distribution_scale	24	0.941100	0.627183
15	Grid	max_depth, aft_loss_distribution_scale	54	0.939963	0.445437
16	Grid	learning_rate, reg_lambda	24	0.941930	0.441468
17	Grid	reg_alpha	6	0.689409	0.404762
18	Grid	min_child_weight, reg_alpha	30	0.692814	0.404762
19	Grid	max_depth, reg_alpha	54	0.689409	0.392857
20	Baseline	None (all defaults)	1	0.013979	0.000000
21	Grid	max_depth	9	0.013979	0.000000
22	Grid	min_child_weight	5	0.013979	0.000000
23	Grid	max_depth, min_child_weight	45	0.013979	0.000000
Dataset H3K36me3_AM_immune, extreme distribution					
0	Baseline	None (all defaults)	1	0.936525	0.902183
1	Grid	learning_rate, reg_lambda	24	0.941356	0.900198
2	Random	All six	100	0.946934	0.898413
3	Grid	min_child_weight	5	0.936525	0.894444
4	Grid	learning_rate	4	0.940548	0.893849
5	Grid	reg_alpha	6	0.938007	0.893254
6	Grid	aft_loss_distribution_scale	6	0.941194	0.890675
7	Grid	reg_lambda	6	0.938167	0.890278
8	Grid	min_child_weight, reg_alpha	30	0.940456	0.889683
9	Grid	min_child_weight, reg_lambda	30	0.940568	0.888095
10	Grid	learning_rate, max_depth	36	0.942076	0.885516
11	Grid	max_depth, aft_loss_distribution_scale	54	0.945163	0.884127
12	Grid	min_child_weight, aft_loss_distribution_scale	30	0.943575	0.883532
13	Random	All six	1000	0.950760	0.882937
14	Grid	learning_rate, min_child_weight	20	0.941308	0.880754
15	Grid	learning_rate, reg_alpha	24	0.942823	0.880159
16	Grid	reg_alpha, reg_lambda	36	0.942876	0.878968
17	Grid	reg_lambda, aft_loss_distribution_scale	36	0.945163	0.878770
18	Grid	reg_alpha, aft_loss_distribution_scale	36	0.945163	0.874008
19	Grid	max_depth, reg_lambda	54	0.942938	0.873810

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
20	Grid	max_depth	9	0.939696	0.867857
21	Grid	learning_rate, aft_loss_distribution_scale	24	0.943575	0.864881
22	Grid	max_depth, min_child_weight	45	0.940518	0.861111
23	Grid	max_depth, reg_alpha	54	0.942137	0.857937
Dataset H3K36me3_TDH_ENCODE, normal distribution					
0	Grid	reg_alpha, reg_lambda	36	0.717657	0.586538
1	Grid	learning_rate, reg_lambda	24	0.719895	0.576923
2	Grid	min_child_weight, reg_alpha	30	0.721049	0.576923
3	Grid	reg_alpha, aft_loss_distribution_scale	36	0.720804	0.567308
4	Grid	min_child_weight, aft_loss_distribution_scale	30	0.728986	0.567308
5	Grid	learning_rate, min_child_weight	20	0.709511	0.557692
6	Grid	max_depth, aft_loss_distribution_scale	54	0.724895	0.557692
7	Random	All six	100	0.758672	0.538462
8	Grid	aft_loss_distribution_scale	6	0.702867	0.528846
9	Grid	reg_lambda	6	0.704511	0.528846
10	Grid	min_child_weight	5	0.709511	0.519231
11	Grid	max_depth, reg_lambda	54	0.713811	0.519231
12	Random	All six	1000	0.775909	0.500000
13	Grid	learning_rate, aft_loss_distribution_scale	24	0.715804	0.500000
14	Grid	reg_lambda, aft_loss_distribution_scale	36	0.732797	0.480769
15	Grid	min_child_weight, reg_lambda	30	0.714511	0.471154
16	Grid	learning_rate, max_depth	36	0.696329	0.461538
17	Baseline	None (all defaults)	1	0.651574	0.451923
18	Grid	learning_rate	4	0.672028	0.442308
19	Grid	reg_alpha	6	0.685874	0.442308
20	Grid	max_depth	9	0.675874	0.403846
21	Grid	max_depth, reg_alpha	54	0.699720	0.394231
22	Grid	learning_rate, reg_alpha	24	0.694021	0.375000
23	Grid	max_depth, min_child_weight	45	0.717203	0.326923
Dataset H3K36me3_TDH_ENCODE, logistic distribution					
0	Grid	min_child_weight, reg_lambda	30	0.737588	0.586538
1	Grid	reg_lambda	6	0.694266	0.557692
2	Grid	aft_loss_distribution_scale	6	0.716049	0.557692
3	Grid	max_depth	9	0.690420	0.548077
4	Grid	reg_alpha, reg_lambda	36	0.713357	0.548077
5	Grid	reg_alpha, aft_loss_distribution_scale	36	0.728741	0.548077
6	Baseline	None (all defaults)	1	0.667727	0.538462
7	Random	All six	100	0.747133	0.538462
8	Grid	learning_rate, reg_lambda	24	0.729895	0.538462
9	Grid	max_depth, reg_lambda	54	0.707658	0.528846
10	Grid	learning_rate, aft_loss_distribution_scale	24	0.729895	0.509615
11	Grid	max_depth, reg_alpha	54	0.690420	0.509615
12	Grid	min_child_weight, aft_loss_distribution_scale	30	0.723741	0.500000
13	Grid	max_depth, aft_loss_distribution_scale	54	0.721049	0.490385
14	Grid	reg_lambda, aft_loss_distribution_scale	36	0.726049	0.490385
15	Grid	reg_alpha	6	0.668636	0.442308
16	Grid	max_depth, min_child_weight	45	0.717658	0.423077
17	Random	All six	1000	0.759371	0.413462
18	Grid	learning_rate, max_depth	36	0.704266	0.394231

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
19	Grid	min_child_weight	5	0.717658	0.394231
20	Grid	learning_rate, min_child_weight	20	0.717658	0.394231
21	Grid	min_child_weight, reg_alpha	30	0.722203	0.394231
22	Grid	learning_rate, reg_alpha	24	0.691574	0.384615
23	Grid	learning_rate	4	0.673182	0.365385
Dataset H3K36me3_TDH_ENCODE, extreme distribution					
0	Baseline	None (all defaults)	1	0.682273	0.548077
1	Grid	max_depth, reg_lambda	54	0.731748	0.538462
2	Grid	max_depth	9	0.700210	0.519231
3	Grid	reg_lambda	6	0.727203	0.519231
4	Grid	min_child_weight, aft_loss_distribution_scale	30	0.736049	0.519231
5	Grid	reg_alpha, reg_lambda	36	0.731748	0.519231
6	Random	All six	100	0.736049	0.509615
7	Grid	aft_loss_distribution_scale	6	0.731049	0.509615
8	Grid	reg_alpha, aft_loss_distribution_scale	36	0.736049	0.509615
9	Grid	learning_rate	4	0.699965	0.500000
10	Grid	learning_rate, min_child_weight	20	0.717902	0.500000
11	Grid	min_child_weight	5	0.700420	0.490385
12	Grid	learning_rate, reg_lambda	24	0.732203	0.480769
13	Grid	learning_rate, reg_alpha	24	0.709965	0.471154
14	Grid	max_depth, min_child_weight	45	0.709511	0.471154
15	Grid	max_depth, reg_alpha	54	0.704511	0.471154
16	Grid	reg_lambda, aft_loss_distribution_scale	36	0.736049	0.471154
17	Grid	learning_rate, max_depth	36	0.704965	0.461538
18	Grid	learning_rate, aft_loss_distribution_scale	24	0.731049	0.442308
19	Grid	max_depth, aft_loss_distribution_scale	54	0.735595	0.432692
20	Random	All six	1000	0.748986	0.403846
21	Grid	reg_alpha	6	0.699965	0.384615
22	Grid	min_child_weight, reg_lambda	30	0.731748	0.384615
23	Grid	min_child_weight, reg_alpha	30	0.714266	0.336538
Dataset H3K36me3_TDH_immune, normal distribution					
0	Grid	min_child_weight, reg_alpha	30	0.925321	0.761905
1	Grid	max_depth, min_child_weight	45	0.937180	0.738095
2	Baseline	None (all defaults)	1	0.870513	0.726190
3	Grid	reg_alpha, reg_lambda	36	0.920833	0.702381
4	Grid	reg_lambda	6	0.912500	0.690476
5	Grid	learning_rate, min_child_weight	20	0.941026	0.690476
6	Grid	min_child_weight, aft_loss_distribution_scale	30	0.940705	0.690476
7	Grid	reg_lambda, aft_loss_distribution_scale	36	0.936859	0.690476
8	Grid	max_depth	9	0.901603	0.678571
9	Grid	reg_alpha	6	0.885577	0.678571
10	Grid	reg_alpha, aft_loss_distribution_scale	36	0.928526	0.678571
11	Grid	max_depth, reg_lambda	54	0.932372	0.666667
12	Grid	max_depth, aft_loss_distribution_scale	54	0.932692	0.666667
13	Grid	learning_rate	4	0.936859	0.654762
14	Grid	aft_loss_distribution_scale	6	0.920833	0.654762
15	Grid	min_child_weight	5	0.921154	0.642857
16	Grid	learning_rate, reg_lambda	24	0.944551	0.642857
17	Grid	min_child_weight, reg_lambda	30	0.941026	0.642857

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
18	Grid	learning_rate, max_depth	36	0.941026	0.630952
19	Grid	learning_rate, reg_alpha	24	0.937180	0.630952
20	Grid	max_depth, reg_alpha	54	0.913141	0.630952
21	Random	All six	100	0.941026	0.607143
22	Random	All six	1000	0.953205	0.571429
23	Grid	learning_rate, aft_loss_distribution_scale	24	0.944872	0.511905
Dataset H3K36me3_TDH_immune, logistic distribution					
0	Grid	reg_alpha, aft_loss_distribution_scale	36	0.941026	0.726190
1	Grid	learning_rate, min_child_weight	20	0.924359	0.714286
2	Grid	learning_rate, reg_alpha	24	0.927885	0.714286
3	Grid	max_depth, aft_loss_distribution_scale	54	0.941026	0.714286
4	Grid	reg_alpha	6	0.924038	0.702381
5	Grid	min_child_weight, aft_loss_distribution_scale	30	0.937180	0.702381
6	Grid	max_depth	9	0.924359	0.690476
7	Grid	min_child_weight, reg_alpha	30	0.924038	0.690476
8	Grid	learning_rate, max_depth	36	0.928205	0.678571
9	Grid	min_child_weight, reg_lambda	30	0.927885	0.678571
10	Grid	max_depth, min_child_weight	45	0.924359	0.666667
11	Baseline	None (all defaults)	1	0.920192	0.666667
12	Grid	learning_rate	4	0.920192	0.666667
13	Grid	min_child_weight	5	0.920192	0.666667
14	Grid	max_depth, reg_lambda	54	0.932372	0.654762
15	Grid	reg_lambda	6	0.927885	0.654762
16	Grid	aft_loss_distribution_scale	6	0.936859	0.654762
17	Grid	max_depth, reg_alpha	54	0.932051	0.654762
18	Grid	reg_alpha, reg_lambda	36	0.927885	0.654762
19	Grid	learning_rate, aft_loss_distribution_scale	24	0.940705	0.630952
20	Grid	reg_lambda, aft_loss_distribution_scale	36	0.941026	0.630952
21	Random	All six	1000	0.956731	0.464286
22	Random	All six	100	0.948398	0.440476
23	Grid	learning_rate, reg_lambda	24	0.932372	0.309524
Dataset H3K36me3_TDH_immune, extreme distribution					
0	Random	All six	1000	0.957051	0.773810
1	Grid	learning_rate, aft_loss_distribution_scale	24	0.940385	0.738095
2	Random	All six	100	0.949039	0.726190
3	Grid	reg_lambda	6	0.932372	0.726190
4	Grid	reg_alpha, aft_loss_distribution_scale	36	0.943910	0.726190
5	Grid	learning_rate, reg_alpha	24	0.932692	0.702381
6	Grid	learning_rate, reg_lambda	24	0.936539	0.690476
7	Grid	reg_alpha, reg_lambda	36	0.940385	0.690476
8	Grid	reg_lambda, aft_loss_distribution_scale	36	0.949039	0.690476
9	Grid	max_depth, reg_lambda	54	0.936859	0.678571
10	Grid	min_child_weight, reg_lambda	30	0.948398	0.666667
11	Grid	learning_rate, max_depth	36	0.941026	0.654762
12	Grid	learning_rate, min_child_weight	20	0.944872	0.642857
13	Grid	min_child_weight, aft_loss_distribution_scale	30	0.944551	0.607143
14	Grid	aft_loss_distribution_scale	6	0.928846	0.607143
15	Grid	max_depth, min_child_weight	45	0.937500	0.607143
16	Grid	max_depth, reg_alpha	54	0.936539	0.607143

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
17	Grid	reg_alpha	6	0.932372	0.595238
18	Grid	max_depth, aft_loss_distribution_scale	54	0.948398	0.595238
19	Grid	min_child_weight, reg_alpha	30	0.944872	0.595238
20	Baseline	None (all defaults)	1	0.916346	0.571429
21	Grid	learning_rate	4	0.932372	0.571429
22	Grid	max_depth	9	0.928846	0.571429
23	Grid	min_child_weight	5	0.925000	0.535714
Dataset H3K36me3_TDH_other, normal distribution					
0	Grid	min_child_weight	5	0.887143	0.765625
1	Random	All six	100	0.926905	0.734375
2	Grid	reg_alpha, reg_lambda	36	0.928095	0.734375
3	Baseline	None (all defaults)	1	0.885952	0.703125
4	Grid	max_depth, min_child_weight	45	0.918571	0.703125
5	Grid	min_child_weight, reg_alpha	30	0.917381	0.687500
6	Random	All six	1000	0.943571	0.671875
7	Grid	reg_alpha	6	0.917381	0.656250
8	Grid	learning_rate, max_depth	36	0.918571	0.656250
9	Grid	max_depth, reg_alpha	54	0.918571	0.656250
10	Grid	max_depth	9	0.908571	0.640625
11	Grid	max_depth, reg_lambda	54	0.918571	0.640625
12	Grid	min_child_weight, aft_loss_distribution_scale	30	0.931667	0.546875
13	Grid	learning_rate, reg_alpha	24	0.917381	0.531250
14	Grid	aft_loss_distribution_scale	6	0.931667	0.515625
15	Grid	reg_lambda	6	0.916190	0.484375
16	Grid	max_depth, aft_loss_distribution_scale	54	0.932857	0.484375
17	Grid	min_child_weight, reg_lambda	30	0.916190	0.484375
18	Grid	reg_alpha, aft_loss_distribution_scale	36	0.932857	0.484375
19	Grid	learning_rate	4	0.909048	0.468750
20	Grid	learning_rate, reg_lambda	24	0.917381	0.468750
21	Grid	learning_rate, aft_loss_distribution_scale	24	0.931667	0.468750
22	Grid	learning_rate, min_child_weight	20	0.909048	0.437500
23	Grid	reg_lambda, aft_loss_distribution_scale	36	0.931667	0.421875
Dataset H3K36me3_TDH_other, logistic distribution					
0	Baseline	None (all defaults)	1	0.918571	0.515625
1	Grid	learning_rate	4	0.931667	0.515625
2	Grid	max_depth	9	0.925714	0.515625
3	Grid	min_child_weight	5	0.918571	0.515625
4	Grid	aft_loss_distribution_scale	6	0.919762	0.515625
5	Grid	learning_rate, max_depth	36	0.932857	0.515625
6	Grid	learning_rate, min_child_weight	20	0.931667	0.515625
7	Grid	learning_rate, aft_loss_distribution_scale	24	0.925714	0.515625
8	Grid	max_depth, min_child_weight	45	0.925714	0.515625
9	Grid	max_depth, reg_alpha	54	0.925714	0.515625
10	Grid	max_depth, aft_loss_distribution_scale	54	0.926905	0.515625
11	Grid	min_child_weight, reg_alpha	30	0.924524	0.515625
12	Grid	min_child_weight, reg_lambda	30	0.918571	0.515625
13	Random	All six	100	0.924524	0.500000
14	Grid	reg_alpha	6	0.924524	0.500000
15	Grid	learning_rate, reg_alpha	24	0.931667	0.500000

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
16	Grid	learning_rate, reg_lambda	24	0.931667	0.484375
17	Grid	reg_alpha, aft_loss_distribution_scale	36	0.925714	0.484375
18	Grid	min_child_weight, aft_loss_distribution_scale	30	0.919762	0.453125
19	Random	All six	1000	0.934048	0.437500
20	Grid	reg_lambda	6	0.918571	0.437500
21	Grid	reg_alpha, reg_lambda	36	0.924524	0.437500
22	Grid	reg_lambda, aft_loss_distribution_scale	36	0.931667	0.437500
23	Grid	max_depth, reg_lambda	54	0.926905	0.406250
Dataset H3K36me3_TDH_other, extreme distribution					
0	Random	All six	1000	0.941190	0.671875
1	Grid	min_child_weight	5	0.909048	0.640625
2	Grid	max_depth, min_child_weight	45	0.909048	0.640625
3	Baseline	None (all defaults)	1	0.886429	0.593750
4	Grid	max_depth	9	0.901905	0.593750
5	Grid	min_child_weight, reg_lambda	30	0.924524	0.578125
6	Grid	reg_alpha	6	0.917381	0.515625
7	Grid	learning_rate, aft_loss_distribution_scale	24	0.931667	0.515625
8	Grid	min_child_weight, reg_alpha	30	0.924524	0.515625
9	Random	All six	100	0.940000	0.484375
10	Grid	learning_rate, min_child_weight	20	0.924524	0.484375
11	Grid	learning_rate	4	0.917381	0.468750
12	Grid	reg_lambda	6	0.917381	0.468750
13	Grid	max_depth, reg_lambda	54	0.924524	0.468750
14	Grid	learning_rate, reg_alpha	24	0.917381	0.453125
15	Grid	learning_rate, max_depth	36	0.924524	0.437500
16	Grid	reg_alpha, reg_lambda	36	0.924524	0.437500
17	Grid	learning_rate, reg_lambda	24	0.917381	0.421875
18	Grid	max_depth, reg_alpha	54	0.924524	0.421875
19	Grid	max_depth, aft_loss_distribution_scale	54	0.931667	0.375000
20	Grid	aft_loss_distribution_scale	6	0.931667	0.359375
21	Grid	min_child_weight, aft_loss_distribution_scale	30	0.931667	0.359375
22	Grid	reg_alpha, aft_loss_distribution_scale	36	0.932857	0.359375
23	Grid	reg_lambda, aft_loss_distribution_scale	36	0.931667	0.281250
Dataset simulated.abs, normal distribution					
0	Baseline	None (all defaults)	1	0.0	0.1375
1	Random	All six	1000	0.0	0.1375
2	Random	All six	100	0.0	0.1375
3	Grid	learning_rate	4	0.0	0.1375
4	Grid	max_depth	9	0.0	0.1375
5	Grid	min_child_weight	5	0.0	0.1375
6	Grid	reg_alpha	6	0.0	0.1375
7	Grid	reg_lambda	6	0.0	0.1375
8	Grid	aft_loss_distribution_scale	6	0.0	0.1375
9	Grid	learning_rate, max_depth	36	0.0	0.1375
10	Grid	learning_rate, min_child_weight	20	0.0	0.1375
11	Grid	learning_rate, reg_alpha	24	0.0	0.1375
12	Grid	learning_rate, reg_lambda	24	0.0	0.1375
13	Grid	learning_rate, aft_loss_distribution_scale	24	0.0	0.1375
14	Grid	max_depth, min_child_weight	45	0.0	0.1375

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
15	Grid	max_depth, reg_alpha	54	0.0	0.1375
16	Grid	max_depth, reg_lambda	54	0.0	0.1375
17	Grid	max_depth, aft_loss_distribution_scale	54	0.0	0.1375
18	Grid	min_child_weight, reg_alpha	30	0.0	0.1375
19	Grid	min_child_weight, reg_lambda	30	0.0	0.1375
20	Grid	min_child_weight, aft_loss_distribution_scale	30	0.0	0.1375
21	Grid	reg_alpha, reg_lambda	36	0.0	0.1375
22	Grid	reg_alpha, aft_loss_distribution_scale	36	0.0	0.1375
23	Grid	reg_lambda, aft_loss_distribution_scale	36	0.0	0.1375
Dataset simulated.abs, logistic distribution					
0	Random	All six	1000	0.875000	0.82500
1	Random	All six	100	0.842187	0.81875
2	Grid	learning_rate, max_depth	36	0.782813	0.76875
3	Grid	learning_rate, aft_loss_distribution_scale	24	0.801562	0.76875
4	Grid	min_child_weight, aft_loss_distribution_scale	30	0.796875	0.75625
5	Grid	max_depth, min_child_weight	45	0.764062	0.73750
6	Grid	max_depth, reg_alpha	54	0.743750	0.73750
7	Grid	max_depth, aft_loss_distribution_scale	54	0.765625	0.72500
8	Grid	learning_rate, min_child_weight	20	0.693750	0.72500
9	Grid	max_depth	9	0.731250	0.70000
10	Grid	learning_rate, reg_alpha	24	0.696875	0.70000
11	Grid	max_depth, reg_lambda	54	0.731250	0.70000
12	Grid	min_child_weight, reg_alpha	30	0.685937	0.70000
13	Grid	aft_loss_distribution_scale	6	0.743750	0.69375
14	Grid	reg_alpha, aft_loss_distribution_scale	36	0.751563	0.68750
15	Grid	reg_lambda, aft_loss_distribution_scale	36	0.745313	0.68125
16	Grid	learning_rate	4	0.684375	0.67500
17	Grid	learning_rate, reg_lambda	24	0.734375	0.66250
18	Grid	min_child_weight, reg_lambda	30	0.689063	0.65625
19	Grid	min_child_weight	5	0.676562	0.65000
20	Grid	reg_alpha	6	0.667188	0.63750
21	Baseline	None (all defaults)	1	0.659375	0.62500
22	Grid	reg_alpha, reg_lambda	36	0.689063	0.55625
23	Grid	reg_lambda	6	0.676562	0.54375
Dataset simulated.abs, extreme distribution					
0	Grid	max_depth, aft_loss_distribution_scale	54	0.868750	0.86250
1	Random	All six	1000	0.896875	0.86250
2	Grid	min_child_weight, aft_loss_distribution_scale	30	0.868750	0.85000
3	Random	All six	100	0.868750	0.83750
4	Grid	reg_alpha, aft_loss_distribution_scale	36	0.843750	0.83750
5	Grid	max_depth, min_child_weight	45	0.834375	0.82500
6	Grid	aft_loss_distribution_scale	6	0.820312	0.81875
7	Grid	learning_rate, aft_loss_distribution_scale	24	0.851562	0.81875
8	Grid	learning_rate, min_child_weight	20	0.820312	0.81250
9	Grid	min_child_weight, reg_lambda	30	0.825000	0.81250
10	Grid	min_child_weight	5	0.815625	0.80000
11	Grid	min_child_weight, reg_alpha	30	0.818750	0.79375
12	Grid	max_depth	9	0.740625	0.78750
13	Grid	max_depth, reg_alpha	54	0.753125	0.78750

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
14	Grid	reg_lambda, aft_loss_distribution_scale	36	0.839063	0.78750
15	Grid	max_depth, reg_lambda	54	0.748437	0.78125
16	Grid	learning_rate, max_depth	36	0.790625	0.76875
17	Grid	learning_rate, reg_lambda	24	0.773438	0.75000
18	Grid	learning_rate	4	0.709375	0.74375
19	Baseline	None (all defaults)	1	0.662500	0.72500
20	Grid	learning_rate, reg_alpha	24	0.715625	0.72500
21	Grid	reg_lambda	6	0.676562	0.68750
22	Grid	reg_alpha, reg_lambda	36	0.690625	0.68125
23	Grid	reg_alpha	6	0.675000	0.67500
Dataset simulated.linear, normal distribution					
0	Baseline	None (all defaults)	1	0.0	0.1
1	Random	All six	1000	0.0	0.1
2	Random	All six	100	0.0	0.1
3	Grid	learning_rate	4	0.0	0.1
4	Grid	max_depth	9	0.0	0.1
5	Grid	min_child_weight	5	0.0	0.1
6	Grid	reg_alpha	6	0.0	0.1
7	Grid	reg_lambda	6	0.0	0.1
8	Grid	aft_loss_distribution_scale	6	0.0	0.1
9	Grid	learning_rate, max_depth	36	0.0	0.1
10	Grid	learning_rate, min_child_weight	20	0.0	0.1
11	Grid	learning_rate, reg_alpha	24	0.0	0.1
12	Grid	learning_rate, reg_lambda	24	0.0	0.1
13	Grid	learning_rate, aft_loss_distribution_scale	24	0.0	0.1
14	Grid	max_depth, min_child_weight	45	0.0	0.1
15	Grid	max_depth, reg_alpha	54	0.0	0.1
16	Grid	max_depth, reg_lambda	54	0.0	0.1
17	Grid	max_depth, aft_loss_distribution_scale	54	0.0	0.1
18	Grid	min_child_weight, reg_alpha	30	0.0	0.1
19	Grid	min_child_weight, reg_lambda	30	0.0	0.1
20	Grid	min_child_weight, aft_loss_distribution_scale	30	0.0	0.1
21	Grid	reg_alpha, reg_lambda	36	0.0	0.1
22	Grid	reg_alpha, aft_loss_distribution_scale	36	0.0	0.1
23	Grid	reg_lambda, aft_loss_distribution_scale	36	0.0	0.1
Dataset simulated.linear, logistic distribution					
0	Grid	learning_rate, aft_loss_distribution_scale	24	0.806250	0.83125
1	Random	All six	100	0.871875	0.81875
2	Grid	max_depth, aft_loss_distribution_scale	54	0.864062	0.81875
3	Grid	reg_lambda	6	0.782813	0.81250
4	Random	All six	1000	0.889062	0.80625
5	Grid	aft_loss_distribution_scale	6	0.800000	0.80625
6	Grid	reg_alpha, reg_lambda	36	0.792188	0.80000
7	Grid	max_depth, reg_lambda	54	0.831250	0.79375
8	Grid	min_child_weight, reg_alpha	30	0.825000	0.79375
9	Grid	reg_alpha, aft_loss_distribution_scale	36	0.817187	0.79375
10	Grid	max_depth	9	0.817187	0.78750
11	Grid	learning_rate, max_depth	36	0.817187	0.78750
12	Grid	learning_rate, reg_lambda	24	0.792188	0.78750

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
13	Grid	min_child_weight	5	0.809375	0.78125
14	Grid	max_depth, reg_alpha	54	0.826562	0.78125
15	Grid	min_child_weight, aft_loss_distribution_scale	30	0.846875	0.78125
16	Grid	max_depth, min_child_weight	45	0.829688	0.77500
17	Grid	reg_lambda, aft_loss_distribution_scale	36	0.840625	0.76875
18	Grid	min_child_weight, reg_lambda	30	0.835938	0.76250
19	Grid	learning_rate, min_child_weight	20	0.817187	0.70625
20	Grid	reg_alpha	6	0.762500	0.70000
21	Grid	learning_rate	4	0.742188	0.68125
22	Grid	learning_rate, reg_alpha	24	0.771875	0.66250
23	Baseline	None (all defaults)	1	0.737500	0.64375
Dataset simulated.linear, extreme distribution					
0	Grid	reg_lambda, aft_loss_distribution_scale	36	0.859375	0.86875
1	Random	All six	1000	0.906250	0.85625
2	Grid	min_child_weight, aft_loss_distribution_scale	30	0.896875	0.85625
3	Grid	min_child_weight, reg_lambda	30	0.870313	0.85000
4	Grid	max_depth, aft_loss_distribution_scale	54	0.889062	0.84375
5	Random	All six	100	0.898438	0.83750
6	Grid	reg_alpha, aft_loss_distribution_scale	36	0.862500	0.83125
7	Grid	aft_loss_distribution_scale	6	0.840625	0.82500
8	Grid	min_child_weight, reg_alpha	30	0.862500	0.82500
9	Grid	max_depth	9	0.854688	0.81875
10	Grid	max_depth, min_child_weight	45	0.867188	0.81875
11	Grid	learning_rate, max_depth	36	0.859375	0.81250
12	Grid	learning_rate, reg_alpha	24	0.800000	0.81250
13	Grid	learning_rate, aft_loss_distribution_scale	24	0.856250	0.80000
14	Grid	max_depth, reg_lambda	54	0.871875	0.80000
15	Grid	reg_lambda	6	0.801562	0.79375
16	Grid	learning_rate, reg_lambda	24	0.812500	0.79375
17	Baseline	None (all defaults)	1	0.768750	0.78750
18	Grid	learning_rate	4	0.768750	0.78750
19	Grid	max_depth, reg_alpha	54	0.862500	0.78750
20	Grid	reg_alpha	6	0.795312	0.77500
21	Grid	min_child_weight	5	0.845313	0.76875
22	Grid	learning_rate, min_child_weight	20	0.850000	0.76875
23	Grid	reg_alpha, reg_lambda	36	0.823438	0.76250
Dataset simulated.sin, normal distribution					
0	Baseline	None (all defaults)	1	0.0	0.325
1	Random	All six	1000	0.0	0.325
2	Random	All six	100	0.0	0.325
3	Grid	learning_rate	4	0.0	0.325
4	Grid	max_depth	9	0.0	0.325
5	Grid	min_child_weight	5	0.0	0.325
6	Grid	reg_alpha	6	0.0	0.325
7	Grid	reg_lambda	6	0.0	0.325
8	Grid	aft_loss_distribution_scale	6	0.0	0.325
9	Grid	learning_rate, max_depth	36	0.0	0.325
10	Grid	learning_rate, min_child_weight	20	0.0	0.325
11	Grid	learning_rate, reg_alpha	24	0.0	0.325

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
12	Grid	learning_rate, reg_lambda	24	0.0	0.325
13	Grid	learning_rate, aft_loss_distribution_scale	24	0.0	0.325
14	Grid	max_depth, min_child_weight	45	0.0	0.325
15	Grid	max_depth, reg_alpha	54	0.0	0.325
16	Grid	max_depth, reg_lambda	54	0.0	0.325
17	Grid	max_depth, aft_loss_distribution_scale	54	0.0	0.325
18	Grid	min_child_weight, reg_alpha	30	0.0	0.325
19	Grid	min_child_weight, reg_lambda	30	0.0	0.325
20	Grid	min_child_weight, aft_loss_distribution_scale	30	0.0	0.325
21	Grid	reg_alpha, reg_lambda	36	0.0	0.325
22	Grid	reg_alpha, aft_loss_distribution_scale	36	0.0	0.325
23	Grid	reg_lambda, aft_loss_distribution_scale	36	0.0	0.325
Dataset simulated.sin, logistic distribution					
0	Grid	min_child_weight, aft_loss_distribution_scale	30	0.850000	0.82500
1	Grid	max_depth, aft_loss_distribution_scale	54	0.853125	0.81875
2	Random	All six	100	0.823438	0.81250
3	Random	All six	1000	0.851562	0.80625
4	Grid	max_depth, reg_alpha	54	0.746875	0.78125
5	Grid	reg_alpha, aft_loss_distribution_scale	36	0.807813	0.77500
6	Grid	reg_lambda, aft_loss_distribution_scale	36	0.804688	0.77500
7	Grid	learning_rate, aft_loss_distribution_scale	24	0.800000	0.77500
8	Grid	aft_loss_distribution_scale	6	0.793750	0.76250
9	Grid	min_child_weight, reg_alpha	30	0.750000	0.75625
10	Grid	learning_rate, max_depth	36	0.745313	0.75000
11	Grid	min_child_weight, reg_lambda	30	0.753125	0.74375
12	Grid	max_depth	9	0.740625	0.73750
13	Grid	learning_rate, min_child_weight	20	0.751563	0.73125
14	Grid	max_depth, reg_lambda	54	0.745313	0.73125
15	Grid	max_depth, min_child_weight	45	0.762500	0.73125
16	Grid	min_child_weight	5	0.742188	0.72500
17	Baseline	None (all defaults)	1	0.643750	0.66250
18	Grid	reg_alpha, reg_lambda	36	0.676562	0.65625
19	Grid	reg_alpha	6	0.654687	0.63750
20	Grid	reg_lambda	6	0.668750	0.63750
21	Grid	learning_rate	4	0.654687	0.62500
22	Grid	learning_rate, reg_lambda	24	0.668750	0.62500
23	Grid	learning_rate, reg_alpha	24	0.676562	0.45625
Dataset simulated.sin, extreme distribution					
0	Grid	max_depth, aft_loss_distribution_scale	54	0.884375	0.87500
1	Grid	reg_lambda, aft_loss_distribution_scale	36	0.865625	0.87500
2	Grid	aft_loss_distribution_scale	6	0.862500	0.86875
3	Grid	reg_alpha, aft_loss_distribution_scale	36	0.862500	0.86875
4	Random	All six	100	0.868750	0.86250
5	Grid	min_child_weight, aft_loss_distribution_scale	30	0.881250	0.85625
6	Random	All six	1000	0.887500	0.85625
7	Grid	learning_rate, aft_loss_distribution_scale	24	0.867188	0.85000
8	Grid	max_depth, min_child_weight	45	0.835938	0.78125
9	Grid	reg_alpha, reg_lambda	36	0.782813	0.78125
10	Grid	max_depth, reg_alpha	54	0.817187	0.77500

(Continued on next page)

Table D.1 (Continued from previous page)

Rank	Method	Hyperparameters selected	# trials	Valid. acc.	Test acc.
11	Grid	min_child_weight, reg_lambda	30	0.834375	0.77500
12	Grid	max_depth	9	0.810937	0.77500
13	Grid	min_child_weight	5	0.829688	0.77500
14	Grid	max_depth, reg_lambda	54	0.815625	0.76875
15	Grid	min_child_weight, reg_alpha	30	0.832812	0.76875
16	Grid	learning_rate, max_depth	36	0.815625	0.76875
17	Grid	learning_rate, min_child_weight	20	0.834375	0.76250
18	Grid	reg_alpha	6	0.776563	0.75625
19	Grid	reg_lambda	6	0.778125	0.75625
20	Baseline	None (all defaults)	1	0.764062	0.75625
21	Grid	learning_rate, reg_lambda	24	0.793750	0.74375
22	Grid	learning_rate	4	0.775000	0.73125
23	Grid	learning_rate, reg_alpha	24	0.782813	0.72500